

TMT-FL: Enabling Trustworthy Model Training of Federated Learning With Malicious Participants

Zhongkai Lu[✉], Lingling Wang[✉], *Member, IEEE*, Zhengyin Zhang[✉], Mei Huang[✉],
Jingjing Wang[✉], *Member, IEEE*, and Meng Li[✉], *Senior Member, IEEE*

Abstract—Federated learning is a widely used method for collaborative machine learning without sharing local data. In this approach, participants train models using their local data, and the model updates are aggregated into a global model. However, ensuring trustworthy model training is crucial because malicious participants may not use their actual local data or may not train the model as intended, which makes it challenging to guarantee the authenticity of the data and the integrity of the model training. To address these issues, we propose a trustworthy model training scheme (TMT-FL) with verifiable authenticity and integrity. Specifically, we leverage zero-knowledge succinct non-interactive argument of knowledge (zk-SNARK) based proofs to verify the integrity of the training execution. To deal with the performance bottleneck in generating zk-SNARK proofs, we use the Chinese Remainder Theorem to optimize the convolution operation, and present an improved zk-SNARK based proof generating scheme which significantly reduces the online proving time. Besides, we adopt matrix commitment along with bloom filter to ensure the authenticity and integrity of the training datasets. Extensive experimental results demonstrate that our improved zk-SNARK scheme performs nearly $3.1\times$ faster than the state-of-the-art in online proving time. Moreover, we experimentally confirm the efficiency of TMT-FL under diverse datasets in terms of computational costs, storage costs, and communication overheads.

Index Terms—Federated learning, trustworthiness, model training, authenticity, integrity, zk-SNARK.

I. INTRODUCTION

FEDERATED learning (FL) [1] is a distributed machine learning framework that is capable of multi-party collaborative training to obtain high-quality global models without sharing data, thereby offering privacy preservation by default. Such a framework can be widely applied in various privacy-sensitive application scenarios, such as medical diagnosis modeling [2], autonomous vehicles [3], smart finance [4], etc. While FL provides

privacy protection for participants, its particular architecture also brings many security issues. Since the model training process of FL is only visible to the participant itself for the sake of privacy protection, it makes FL vulnerable to malicious participants. Take a cross-silo FL in the medical scenario as an example, each medical institution has a vast amount of medical data and they use these data locally to train a global model collaboratively and then get rewards according to the amount of data they have used. However, some dishonest institutions may only use part of their local data to update the local model rather than all the data they claimed, or even submit outdated/fake model updates without executing local training tasks as intended. And then, they try to defraud rewards with fewer datasets and training efforts. Obviously, these misbehaviors will affect the correctness of the local model, damage model accuracy and impede the global model convergence. Especially, it may lead to untimely or incorrect decision-making in the FL-based applications, which will ultimately cause severe economic loss.

Regarding these issues, there are many defense solutions to prevent data/model poisoning attacks from unreliable participants [5], [6], [7] or remove anomalous model updates [8] to guarantee the convergence of the global model. Unfortunately, none of them takes into account whether the real claimed data is used in the model training process under an FL scenario with malicious participants, i.e., the authenticity and integrity problem of the datasets. Moreover, the training integrity problem, i.e., whether the entire neural network model is executed as intended, is also an open problem in the untrustworthy FL scenario. To our best knowledge, few works have explored these problems. In this paper, we aim to build a trustworthy model training scheme (TMT-FL) that provides data authenticity, data integrity, and training integrity guarantees.

Data authenticity indicates that participants indeed use their real local data to train models. To verify the authenticity of the training subsets, one traditional solution is to commit the subset for each training round via Pedersen commitments [9]. However, it results in significant computational cost and communication overhead, since the size of the commitments which are computed on the group is bigger than that of the raw dataset. In addition, data integrity requires that participants use the entire claimed local data in the FL task. One straightforward solution is to compute a hash value of the local data [10]. However, it can not work in the model training of FL since the training subset of each round is a different subset of the local data. Hence, new solutions are urgently needed to balance efficiency and availability.

Received 7 February 2024; revised 17 October 2024; accepted 12 December 2024. Date of publication 23 December 2024; date of current version 15 May 2025. This work was supported in part by NSFC under Grant 61802217, Grant 62372149, and Grant U23A20303, in part by NSFSD under Grant ZR2023MF082, in part by Qingdao Science and Technology Plan Key Research and Development Project under Grant 22-3-4-xxgg-10-gx, and in part by Original Exploration Project of Qingdao NSF under Grant 23-2-1-164-zyyd-jch. (Corresponding author: Lingling Wang.)

Zhongkai Lu, Lingling Wang, Zhengyin Zhang, Mei Huang, and Jingjing Wang are with the School of Information Science and Technology, Qingdao University of Science and Technology, Qingdao, Shandong 266061, China (e-mail: luzhongkai@mails.qust.edu.cn; wanglingling@qust.edu.cn; ZhangZhengyin@mails.qust.edu.cn; huangmei@mails.qust.edu.cn; wangjingjing@qust.edu.cn).

Meng Li is with the School of Computer Science and Information Engineering, Hefei University of Technology, Hefei, Anhui 230601, China (e-mail: mengli@hfut.edu.cn).

Digital Object Identifier 10.1109/TDSC.2024.3521377

Training integrity shows that participants execute the model training correctly as intended. It enables fair and correct model updates by preventing lazy participants from incompletely performing the training process or malicious participants from providing incorrect model updates. Existing works usually implement the integrity verification of the training execution in two ways: TEE (Trusted Execution Environments)-based [11] and zk-SNARK proof-based [12] solution. Since it is difficult to meet a unified trusted execution environment for all participants with heterogeneous hardware resources, the proof-based solution is a good alternative. However, it leads to extreme performance degradation if directly generating proof based on classic zk-SNARK [13]. Since the complicated and huge calculations of the zk-SNARK consume most of the time on creating proofs, the proving time becomes the performance bottleneck of the zk-SNARK based proof [14], [15], [16]. Therefore, it is critical to reduce the proving overhead of generating zk-SNARK based proofs.

To address the above issues, we envision guaranteeing the trustworthy model training process by optimizing the classic zk-SNARK and devising new strategies to generate trusted proofs and verify the authenticity and integrity of the training datasets with matrix computation. Though promising, it is still a nontrivial task. First, it is challenging to optimize the proving overhead for convolution operations that dominate the performance bottleneck. Since there are numerous multiplicative gates involved in one convolutional operation and meanwhile the model training contains a vast number of convolutional operations, these lead to plenty of multiplications on the Quadratic Arithmetic Programs (QAP)-based zk-SNARK. Second, the model training process consists of multiple rounds of neural network computations and each round handles a different subset of the local data, so data integrity verification is challenging because the number of the training subsets and the training rounds are not fixed. It is not easy to find an efficient mapping between the local dataset and the training subsets without compromising local data privacy.

For the first challenge, we devise an “offline-then-online” proof generating strategy by partitioning the proof generation into an offline part of datasets preprocessing executed locally by the participant, and an online part involving an improved zk-SNARK based proof generation scheme. Then the online proving time of the participant can be reduced significantly which is critical for the online convergence speed of the global model. For the offline operations, we propose to optimize the convolution computations by using the Chinese Remainder Theorem, and we also make batch proofs for convolutional layer operations. Moreover, we modify the Quadratic Matrix Programs (QMP)-based zk-SNARK in order to match our design. For the second challenge, we propose a strategy that handles the division of local subsets corresponding to the training rounds. Considering all possible cases, we give a data integrity verification scheme by constructing a lightweight matrix commitment scheme along with bloom filters to guarantee computational cost and query efficiency.

To summarize, the main contributions of our work include:

- We propose a trustworthy model training scheme TMT-FL, which appropriately integrates improved zk-SNARK,

matrix commitments and bloom filters, to guarantee the data authenticity, training integrity and data integrity in an FL system with malicious participants.

- We optimize the proving time of the zk-SNARK based proof by devising an “offline-then-online” proof-generating strategy and modifying the QMP-based zk-SNARK. Our scheme can simultaneously achieve the verifiability of both the training process and the model updates.
- We give a comprehensive security analysis to prove the security of TMT-FL. Besides, the commitments of training subsets and server-side verification are both processed with the lightweight technique. Extensive experiments conducted on real-world datasets demonstrate that TMT-FL achieves high performance in terms of computational costs, storage costs and communication overheads and does not affect model accuracy.

The remainder of this paper is organized as follows. In Section II, we present the system model, threat model and assumptions, and design goals. Some preliminaries are in Section III. In Section IV, we propose our scheme. Subsequently, security analysis and performance evaluation are presented in Sections V and VI. Then, we review some related works in Section VII and give some discussions about TMT-FL in Section VIII. Finally, Section IX draws the conclusion.

II. PROBLEM STATEMENT

In this section, we formalize the system model, threat model, and present our design goals.

A. System Model

We focus on cross-silo federated learning where a few organizations have large amounts of private data. Each organization is assigned a unique identifier for indexing, whose dataset samples are independently distributed and share the same feature space. They are capable of participating in FL tasks with a low probability of failure. Then, we divide the entities of TMT-FL into the task initiator, participants, and the aggregation server, which interact as shown in Fig. 1.

Task initiator can be institutions providing medical, automobile, or financial services. These institutions autonomously publish FL tasks according to their demands and resources, and generate public parameters.

Participants act as organizations with personalized datasets that they don't want to reveal to anyone else. Let $\{P_1, P_2, \dots, P_i, \dots\}$ be participants with the datasets $\{D_1, D_2, \dots, D_i, \dots\}$ respectively. They also have different resources for computation, communication, and storage. At each training round, participants download the latest global model, update the local model using their claimed local datasets, and do not drop out halfway. They also provide trusted proofs to prove that the model training is executed faithfully.

Aggregation Server (AS) acts as the aggregator of FL tasks. The AS maintains a bloom filter, which is used to verify the authenticity and integrity of the participants' training datasets. When the training subsets are authentic and the trusted proofs

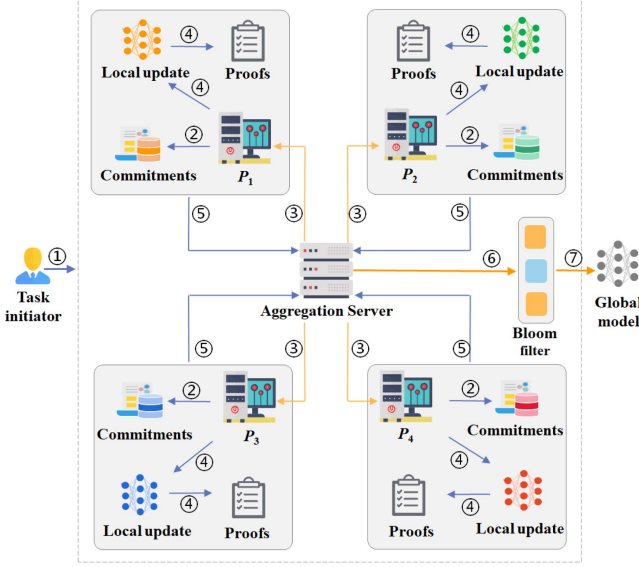


Fig. 1. System model.

are verified successfully, the AS aggregates participants' trustworthy model updates and publishes the new global model.

The workflow of TMT-FL is as follows: ① The task initiator publishes an FL task and generates public parameters. ② Participants download the public parameters to preprocess the datasets. ③ The AS publishes the global model. ④ Participants perform the local model training to obtain model updates and generate proofs based on the training processes. ⑤ Participants packet proofs and encrypted model updates, and transmit them to the AS. ⑥ The AS receives messages from participants, makes queries to the bloom filter and verifies the proofs. ⑦ The AS performs global model aggregation to obtain the new global model. Repeat steps ③ to ⑦ until the global model converges within the required accuracy after multiple training rounds.

The key notations are listed in Table I.

B. Threat Model

We consider that the task initiator and the AS are semi-honest. It means that the task initiator coordinates the whole FL procedure faithfully, desires the aggregated global model trained by participants, and it stays offline after the public parameters generation. The AS properly performs the authenticity and integrity verification of datasets and verifies the proofs generated by each participant. Also, the task initiator and the AS are curious about the sensitive data of the involved participants, and they passively launch privacy attacks to obtain participants' privacy from the local model updates. The attacks stealing the raw dataset indirectly through inferred global models are beyond the scope of this paper.

Participants are untrusted in our model, and they may deviate from the procedure of the model training or be maliciously manipulated during the FL task. Specifically, they may 1) use poisoned datasets or parts of the datasets to train the local model; 2) return arbitrary model updates without executing the whole procedure of the model training; 3) launch free-riding attacks,

TABLE I
KEY NOTATIONS

Notation	Definition
N	the number of sample matrices per round
M	the number of kernels
$n \times n$	the dimension of the sample matrix
n	the number of the participants
$m \times m$	the dimension of the kernel
P_i	the i -th participant
D_i	the local dataset of the participant P_i
$\vec{g}$	the gradient vector of a participant
pp	the public parameters
g, h	the generators of G_1, G_2
$\langle X_i, Y_i \rangle$	the input matrix and its label
∇L_f	the derivative of the loss function
ϵ	the false positive rate
λ	the security parameter
π	the trusted proof of the local model training
θ	the global model
crs	common reference string in zk-SNARK
ϕ	the statement in zk-SNARK
ω	the witness in zk-SNARK
U	the singular square matrix
E	the set of sample matrices
W	the set of kernel matrices
Z	the set of feature matrices
Q	uncertain variables in zk-SNARK
PM, RM	operation matrices for pooling and Relu
$\ \cdot\ _2$	L_2 norm

i.e., submit incorrect model updates with no training efforts; use one training effort to claim multiple rewards.

C. Design Goals

Our design goal is to implement an FL scheme with a trustworthy model training guarantee. It is capable of the authenticity and integrity verification of the training datasets, and the integrity verification of the model training process. It also has the ability to defend against some security attacks.

Data authenticity: The dataset used in each round is a subset of the real local data committed before the FL task.

Training integrity: TMT-FL provides the integrity of the model training execution by effectively verifying the trusted proof which provides completeness, soundness, and zero-knowledge guarantee. The local updates generated without executing the whole training process will be detected and discarded in the model aggregation step.

Data integrity: TMT-FL can confirm that the entire claimed local data has been used in an FL task. The behavior of using only part of the claimed data rather than the entire data when performing the FL task can be easily detected.

Resistance to free-riding attacks: TMT-FL resists free-riding attacks by data integrity and training integrity verification. Lazy participants who launch free-riding attacks cannot pass the integrity verification.

Privacy preservation: TMT-FL protects participants' local model updates against the semi-honest AS and the malicious participants, i.e., they cannot obtain any data privacy by inferring the encrypted local model updates and proofs of the participants.

III. PRELIMINARIES

In this section, we introduce Neural Network Model, zk-SNARK, Chinese Remainder Theorem, and Bloom Filter that will be adopted to construct our scheme.

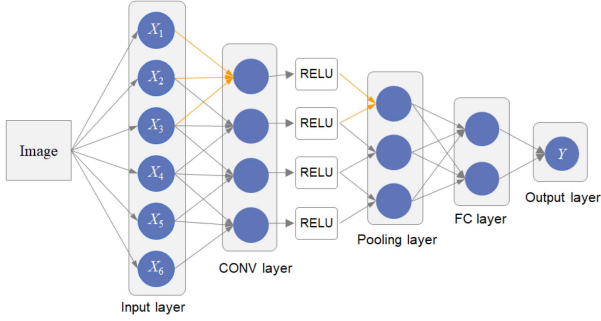


Fig. 2. An example of a convolutional neural network.

A. Neural Network Model

In federated learning, a neural network model is trained collaboratively by aggregating local models generated by multiple participants using their local data [1]. We adopt the convolutional neural network (CNN) in TMT-FL as the local training model. CNN is a class of feedforward neural networks with representation learning capability and has spawned a variety of network structures that serve different scenarios. Fig. 2 shows a simple CNN model which contains a convolutional layer, an activation layer, a pooling layer, and a fully connected layer. CNN models can be built with more complex network structures by stacking multiple convolutional and non-linear layers.

As described in Fig. 2, the input samples are mapped to different classifications by operations $\hat{Y} = f^{fc}(f^{pool}(f^{act}(f^{conv}(X, \theta))))$ layer by layer, where θ is the parameters. The convolution operation is to extract features of the input sample which is represented as an input matrix. Given an input $D = \{< X_i, Y_i > | i = 1, \dots, N\}$, convolution layers would output $Z = f^{conv}(X, W)$, where W is the *kernels*. Then, activation layer is used to map linear features Z to high-dimensional non-linear intervals $\hat{Z}$, where $\hat{Z} = f^{act}(Z)$. The pooling layer is used to downsample the data passed from the previous layer in CNN models, which reduces information redundancy while retaining the essential features of data. And it obtains $\tilde{Z} = f^{pool}(\hat{Z})$ as the input of the full connection layer, in which $\tilde{Z}$ multiplies by W to capture the original features of input images more globally.

The loss function of CNN can be defined as follows:

$$L_f(D, \theta) = \frac{1}{T} \sum_{(X_i, Y_i) \in D'} (Y_i - f(X_i, \theta))^2.$$

In our scheme, we obtain the gradient based on the stochastic gradient descent (SGD): $\tilde{g}_i \leftarrow \nabla L_f(D', \theta)$, where ∇L_f is derivative of the loss function L_f and D' is a random subset of D .

B. Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge

The zero-knowledge succinct non-interactive arguments of knowledge (zk-SNARK) [13] is a novel form of non-interactive zero-knowledge (NIZK) proofs. It allows a prover to convince a verifier that he knows some secret information without leaking any useful knowledge, and the proof can be

verified in a few milliseconds. A zk-SNARK algorithm $ZS = (Setup, Prove, Verify, Sim)$ is composed of the following probabilistic polynomial time (PPT) algorithms:

- *Setup*(R): input the constraint relations R , output the common reference string crs and the trapdoor τ . R is the non-interactive linear proof output constructed by the quadratic arithmetic program generated by the constraint condition.
- *Prove*(crs, ϕ, ω): input a common reference string crs , a statement ϕ and a witness ω , return a proof π .
- *Verify*(crs, ϕ, π): input a common reference string crs , a statement ϕ and an argument π , return a bit b .
- *Sim*(R, τ, ϕ): input the constraint relations R , a simulation trapdoor τ and statement ϕ , return an argument π .

C. Chinese Remainder Theorem

Chinese Remainder Theorem (CRT) [17] states that if one knows the remainders of the euclidean division of an integer x by several integers, then one can determine the remainder of the division of x uniquely by the product of these integers, under the condition that the divisors are pairwise coprime:

$$\begin{cases} x \equiv a_1 \pmod{h_1} \\ x \equiv a_2 \pmod{h_2} \\ \vdots \\ x \equiv a_n \pmod{h_n} \end{cases}$$

As the above equations show, it can speed up computation for x by calculating the cumulative multiplication of n moduli as $\Delta = \prod_{i=1}^n h_i$ and its inverse. Specifically,

$$x \equiv \sum_{i=1}^n a_i \times \frac{\Delta}{h_i} \times \left[\left(\frac{\Delta}{h_i} \right)^{-1} \right]_{h_i} \pmod{\Delta}.$$

CRT is widely used for computing with large integers, as it allows replacing a computation for which one knows a bound on the size of the result by several similar computations on small integers.

D. Bloom Filter

A bloom filter is a space-efficient probabilistic data structure conceived by Burton Howard Bloom in 1970 [18], and used to test whether an element is a set member. Bloom proposed the technique for the situation that conventional error-free hashing techniques lead to heavy storage overheads. Specifically, an empty bloom filter is a bit array of L bits, all set to 0. There must also be ξ different hash functions defined, each of which maps or hashes some set element to one of the L array positions, generating a uniform random distribution. Typically, k is a small constant that depends on the desired false positive rate ϵ , while L is proportional to k and the number of elements to be added.

IV. DETAILED DESIGN

In this section, we first provide the technical overview that underpins the proposed TMT-FL and then present the detailed procedure, i.e., system initialization, trusted proof generation,

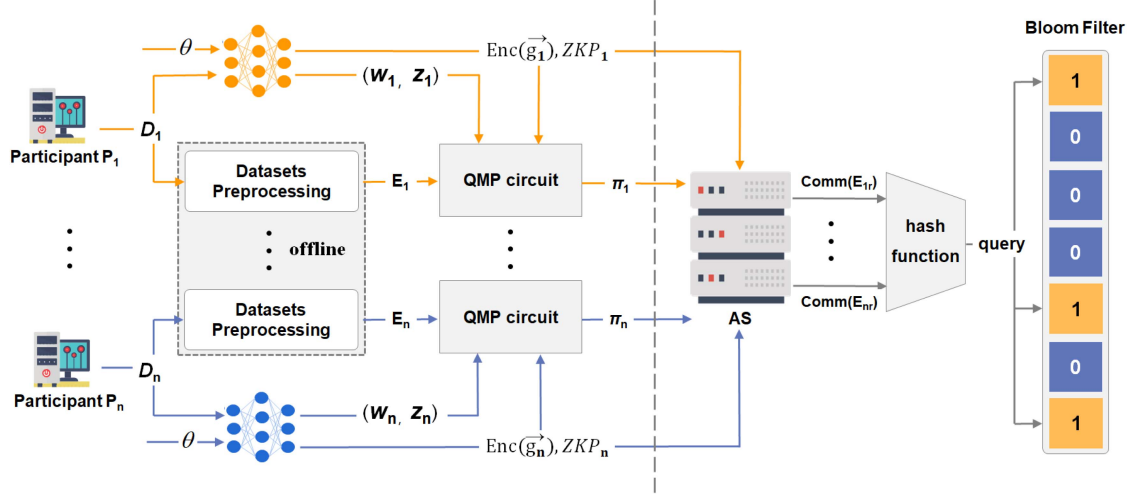


Fig. 3. The data flow of TMT-FL.

and trustworthiness verification, as shown in Algorithm 1. The data flow is also demonstrated in Fig. 3.

A. Technical Overview

We propose an “offline-then-online” proof-generation strategy to optimize the proving time. In the offline phase, the datasets preprocessing is executed locally by the participants by using *CRT* to package and compress the local datasets, thereby reducing the amount of data processed by the convolutional layer when generating the proof (See Algorithm 2). Since our design takes advantage of QMP-based zk-SNARK to generate the trusted proof, the *im2col* operation is adopted to convert convolutional operations to matrix multiplication. For the data authenticity and data integrity verification, matrix commitments $Comm(\cdot)$ are leveraged to make commitments to the local datasets.

As for the online trusted proof generation, we assume the common reference string CRS has been generated in advance. We modify the QMP-based zk-SNARK and devise an improved zk-SNARK based proof generation algorithm (See Algorithm 3). Meanwhile, we make batch matrix multiplication to generate proofs of convolutional operations to improve the efficiency. For the non-linear layers of the CNN, we first convert the non-linear operations to the matrix operations and then use the proposed algorithm to generate the proof. Hence, we can prove the matrix multiplication of each layer with the improved algorithm, and then adopt CP-SNARKs [19] to guarantee consistency between adjacent layers, which outputs the final trusted proof. Moreover, in order to solve the problem of privacy protection and correctness verification, the local model updates are encrypted and the corresponding NIZK proofs are generated, which allow the AS to verify the correctness of the encrypted local model updates.

Before the trustworthiness verification, incorrect model updates will be first filtered out by verifying the NIZK proofs, which help resist the model poisoning attacks caused by the poisoned datasets. After that, the AS executes the trustworthiness verification (See Algorithm 4) to check: ① whether the normal model updates are obtained by executing the entire local model

training faithfully, and ② whether the participants have used all their claimed datasets to train the model. For task ①, a training integrity verification is proposed to verify the trusted proof, which can pass the verification only if it is generated according to algorithm 3. For task ②, the data authenticity will be verified at each round, and the data integrity is only verified after all training rounds. Bloom filter is used to verify the authenticity of data subsets. As long as the commitment of this subset has been stored in the bloom filter by the AS in the initialization phase, the subset can pass the data authenticity verification. As for the data integrity verification, we present a verification equation by leveraging the homomorphic and binding properties of the matrix commitment. Only the participant who has used all their claimed data subsets can make the verification equation hold.

B. System Initialization

The task initiator distributes a training task and the expected number of training rounds T , and selects a group $\mathbb{Z}_p^*$ of prime order $p > 2^\lambda$, where λ is the security parameter, a bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, where $\mathbb{G}_1, \mathbb{G}_2$ are multiplicative cyclic groups of prime order p , and $g \in \mathbb{G}_1, h \in \mathbb{G}_2$ are the generators, and the collision-resistant hash function $hash : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$. The task initiator also specifies the dimension of the sample matrix and the kernel as $n \times n$ and $m \times m$, constructs a $m^2 \times m^2$ singular square matrix U and a modulus list $\{h_1, h_2, \dots, h_{n-m+1}\}$. Then, the public parameters are

$$pp = \{T, \mathbb{Z}_p^*, g, h, hash, n, m, U, \{h_1, h_2, \dots, h_{n-m+1}\}\}.$$

In order to generate and verify the zk-SNARK based proof, the CRS needs to be generated in advance. In our scheme, the task initiator can ask for a trusted third party to generate the CRS as in the work [13]. In order to reduce the reliance on the trusted third party, the task initiator can also select ρ participants to generate the CRS collaboratively [12]. Finally, the CRS is calculated as

Algorithm 1: TMT-FL.

Input: Security parameter, local datasets, and global model;
Output: System parameters, trusted proofs, data commitments, and trustworthy global model updates.

/ System Initialization */*

- 1: The task initiator generates system parameters with security parameters, publishes the FL task, and assists the common reference string (CRS) generation;
- 2: The participants perform a task, download the parameters, preprocess their local datasets, and then submit the commitments of the claimed datasets to the AS;
- 3: The AS receives participants' commitments, maintains a bloom filter for querying and publishes global model θ .

/ Trusted Proof Generation */*

- 4: The participant selects and uses one committed subset to update the local model, and generates a trusted proof of local training execution based on our improved proof generation algorithm;
- 5: The participant encrypts the model update and generates an NIZK proof for the encrypted model update;
- 6: The participant signs the message including the trusted proof, the encrypted model update, and the NIZK proof, and sends the message along with the signature to the AS.

/ Trustworthiness Verification */*

- 7: In each training round, the AS parses participants' messages, filters out incorrect model updates, queries the bloom filter to verify the data authenticity. If the committed dataset can be found in the bloom filter, it then verifies the trusted proof;
- 8: If the above proof is valid, the AS aggregates and updates the global model of this round.
- 9: When the global model converges, the AS performs data integrity verification.

$crs = (g^{\sigma_1}, h^{\sigma_2})$, where

$$\sigma_1 = \left\{ \alpha, \beta, \delta, \left\{ x^i \cdot z^j \right\}_{i=0, j=0}^{n^2, n}, \left\{ \frac{\beta L_i(x) + \alpha R_i(x) + O_i(x)}{\gamma} \right\}_{i=0}^s, \left\{ \frac{\beta L_i(x) + \alpha R_i(x) + O_i(x)}{\delta} \right\}_{i=s+1}^N, \left\{ \frac{x^i t(x)}{\delta} \right\}_{i=0}^{n^2-1} \right\},$$

$$\sigma_2 = \left\{ \beta, \gamma, \delta, \left\{ x^i \cdot z^j \right\}_{i=0, j=0}^{n^2, n} \right\},$$

$\alpha, \beta, \gamma, \delta, x, z$ are the trapdoor, $L_i(x), R_i(x), O_i(x), t(x)$ are the polynomials, s is the length of statements, and N is the number of samples. This prevents the leakage of the trapdoor during the CRS generation. In case some selected participants are malicious, we can use the verifiable secret sharing [20] and masking techniques to verify the parameters shared by the participants. Due to the space limitations, we provide the details of the CRS generation in our submitted *Supplemental Materials*.

Assume a participant P_j downloads the public parameters pp and divides the local data D_j into T subsets $\{D_{j1}, D_{j2}, \dots, D_{jT}\}$, each of which contains N samples in the form $\langle X_i, Y_i \rangle$. Then, according to Algorithm 2, the

Algorithm 2: Local Datasets Preprocessing.

Input:
Sample matrix X_i with the dimension $n \times n$;
Modulus_list $\{h_1, h_2, \dots, h_{n-m+1}\}$;

Output:
Processed matrix E_i ;

- 1: The P_j calculates and gets parameter matrix X'_i via CRT with X_i as the remainders and H as the pairwise coprime moduli;
- 2: **for** $k = 1$ to n **do**
- 3: P_j selects $(x_{1,k}, x_{2,k}, \dots, x_{n,k})^T$, then
- 4: **for** $t = 1$ to m **do**
- 5: Executes
- 6: $e_{t,k} = CRT(x_{t,k}, x_{t+1,k}, \dots, x_{t+n-m,k})$
- 7: with $\{h_1, h_2, \dots, h_{n-m+1}\}$ as the moduli;
- 8: Obtains column vectors $(e_{1,k}, e_{2,k}, \dots, e_{m,k})$;
- 9: **end for**
- 10: Gets parameter matrix X'_i with elements $\{e_{t,k}\}$;
- 11: **end for**
- 12: $E_i = im2col(X'_i)$;
- 13: Return E_i .

P_j preprocesses the sample matrix $D_{ji} = \{X_1, X_2, \dots, X_N\}$, $i \in \{1, \dots, T\}$ by using the CRT and $im2col$ operation [21] as shown below.

$$X_i = \begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,n} \\ x_{2,1} & x_{2,2} & \dots & x_{2,n} \\ \dots & \dots & \dots & \dots \\ x_{n,1} & x_{n,2} & \dots & x_{n,n} \end{bmatrix}$$

$$\xrightarrow{CRT \downarrow}$$

$$X'_i = \begin{bmatrix} e_{1,1} & e_{1,2} & \dots & e_{1,n} \\ e_{2,1} & e_{2,2} & \dots & e_{2,n} \\ \dots & \dots & \dots & \dots \\ e_{m,1} & e_{m,2} & \dots & e_{m,n} \end{bmatrix}$$

$$\xrightarrow{im2col \downarrow}$$

$$E_i = \begin{bmatrix} e_{1,1} & e_{1,2} & \dots & e_{1,n-m+1} \\ e_{2,1} & e_{2,2} & \dots & e_{2,n-m+1} \\ \dots & \dots & \dots & \dots \\ e_{m^2,1} & e_{m^2,2} & \dots & e_{m^2,n-m+1} \end{bmatrix}$$

After datasets preprocessing, the subset D_{ji} is converted to $E_{ji} = \{E_1, E_2, \dots, E_N\}$. And the dataset D_j is represented in the corresponding form $E_j = \{E_{j1}, E_{j2}, \dots, E_{jT}\}$. Then, the P_j makes commitments of the local data E_j and the subset E_{ji} , noted as $Comm(E_j) = U \times E_j$ and $Comm(E_{ji}) = U \times E_{ji}$, and sends them to the AS. The AS computes $\eta = hash(comm(E_j))$ and saves it. Then, the AS selects a large prime q randomly and ξ collision-resistant hash functions $H_1(\cdot), H_2(\cdot), \dots, H_\xi(\cdot)$, and constructs a bloom filter of length L with the false positive rate ϵ , to which the hash of all commitments will be deposited.

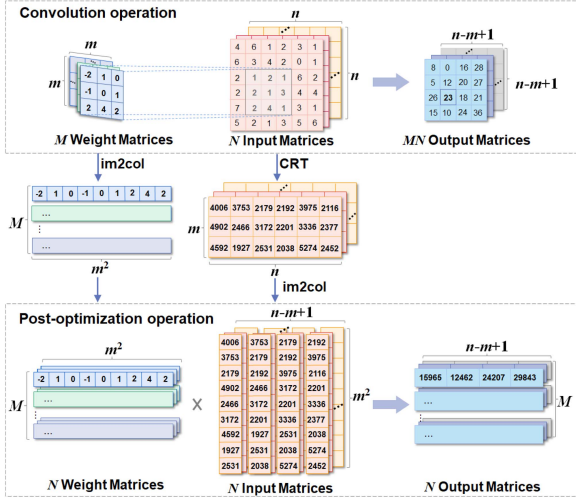


Fig. 4. Matrix multiplication optimization for convolution operations.

C. Trusted Proof Generation

Assume the P_j parses the global model θ_r to obtain the kernel $\mathbf{W}$ and the bias in the r -th round, selects a committed subset $D_{jr} = \{X_1, X_2, \dots, X_N\}$ to train the local model. After the local model training, the kernel $\mathbf{W}$ is updated and noted as $\mathbf{W}_{jr} = \{W_1, W_2, \dots, W_N\}$. Then, P_j extracts and records the feature matrix $\mathbf{Z}_{jr} = f^{conv}(X_i, \mathbf{W}_{jr})$, $i \in \{1, \dots, N\}$, which is used to generate zk-SNARK proofs.

1) *Convolutional Relations Optimization*: Assume the P_j has performed Algorithm 2 in batch on the sample matrices D_{jr} to obtain $\mathbf{E}_{jr}$, and there are M kernels of the dimension $m \times m$ in a training round. We first follow the *im2col* algorithm to convert the dimension of the kernel from $m \times m$ to $1 \times m^2$. After that, we can get an $M \times m^2$ matrix from M kernels. And then, the convolution operations can be converted to matrix multiplication operations, as shown in Fig. 4.

The feature map, i.e., the output matrix of the convolution operation, is a matrix of $(n - m + 1) \times (n - m + 1)$ obtained by convolving the kernel of $m \times m$ on sample matrix of $n \times n$. We perform CRT on each column of the feature maps to obtain M vectors of the dimension $1 \times (n - m + 1)$. Then we get a feature matrix $\mathbf{Z}_{jr}$ of $M \times (n - m + 1)$, where $\mathbf{Z}_{jr} = \{Z_1, Z_2, \dots, Z_N\}$. Since the participant P_j is untrusted in our threat model, we need to verify whether P_j has actually executed the convolution operations. This can be achieved by checking whether the matrix multiplication equation $\mathbf{W}_{jr} \times \mathbf{E}_{jr} = \mathbf{Z}_{jr}$ holds.

Since $\mathbf{W}_{jr}$, $\mathbf{E}_{jr}$ and $\mathbf{Z}_{jr}$ have different dimensions, we extend them into the same dimension of $n \times n$ by padding "0", such that $\mathbf{W}_{jr} \times \mathbf{E}_{jr} = \mathbf{Z}_{jr}$ still holds. There are N matrix multiplication equations, which represent all convolution operations for each layer per iteration. And we add all equations as follows:

$$\sum (W_i \times E_i) = \sum Z_i,$$

where $i \in \{1, \dots, N\}$. Here, in order to reduce the computational cost of matrix multiplications, we transform the above equation into the form $\sum W_i \times \sum E_i = \sum Z_i$ by introducing uncertain variables $\{Q_1, Q_2, \dots, Q_N\}$ where Q_i

is a $n \times n$ -dimensional non-singular matrix that satisfies the following equation:

$$\sum Q_i W_i \times \sum Q_i E_i = \sum Q_i Z_i.$$

According to QMP definition [22], a QMP consists of three sets of polynomials $\{L(x), R(x), O(x)\}$ and a target polynomial $t(x)$. When the matrix multiplication computation is expressed by arithmetic circuits over matrices, the corresponding QMPs of these matrices on arithmetic circuits are of the form $\{L_{\mathbf{W}}(x), R_{\mathbf{E}}(x), O_{\mathbf{Z}}(x), t(x)\}$. For the statement $(Q_1, Q_2, \dots, Q_s)$ and the witness $(Q_{s+1}, Q_{s+2}, \dots, Q_N)$, there exists a non-singular matrix μ with the dimension $n \times n$. For $z \in Z_p^*$, it is required that $t(x)$ divides $p(x, z)$, where

$$p(x, z) = \text{tr} \left\{ \mu \cdot \sum_{i=0}^N Q_i(z) L_i(x) \cdot \sum_{i=0}^N Q_i(z) R_i(x) \right\} - \text{tr} \left\{ \mu \cdot \sum_{i=0}^N Q_i(z) O_i(x) \right\}.$$

Then, the multiplication gate of the QMP is constrained by the following condition:

$$\begin{aligned} & \text{tr} \left\{ \mu \cdot \sum_{i=0}^N Q_i(z) L_i(x) \cdot \sum_{i=0}^N Q_i(z) R_i(x) \right\} \\ &= \text{tr} \left\{ \mu \cdot \sum_{i=0}^N Q_i(z) O_i(x) \right\}. \end{aligned}$$

The left input wire of the multiplication gate is labeled by $\sum_{i=0}^N Q_i(z) L_i(x)$, the right input wire is labeled by $\sum_{i=0}^N Q_i(z) R_i(x)$, and the output wire is $\sum_{i=0}^N Q_i(z) O_i(x)$.

Note that we combine the matrix commitment equation $\mathbf{U} \times \mathbf{E}_{jr} = \text{Comm}(\mathbf{E}_{jr})$ with the convolution equation $\mathbf{W}_{jr} \times \mathbf{E}_{jr} = \mathbf{Z}_{jr}$ to generate the trusted proof in order to ensure the consistency between the committed subset and the training subset, as shown in Algorithm 3.

According to Algorithm 3, P_j parses ϕ and ω as $(Q_1, Q_2, \dots, Q_s)$ and $(Q_{s+1}, Q_{s+2}, \dots, Q_N)$, selects $\psi, s \xleftarrow{\$} Z_p^*$, and calculates $A, B, C, D, \text{Comm}(\mathbf{E}_{jr})$ to get the proof of the convolutional layer π_{cov} , which makes the equation $e(A, B) = e(g^\alpha, h^\beta) \cdot e(D, h^\gamma) \cdot e(C, h^\delta) \cdot e(g^{\phi \text{Comm}(\mathbf{E}_{jr})}, h^\gamma)$ hold.

2) *Pooling and Activation Operations*: We adopt the arithmetic convert methods [23] to transform non-linear operations to matrix operations. For the activation layer, we construct the ReLU matrix $\mathbf{RM}$ with "0" and "1", where "1" corresponds to the elements that are greater than zero in $\mathbf{Z}$. We perform matrix multiplications rather than activation functions on $\mathbf{RM}$ and the feature matrix $\mathbf{Z}$ which is the output matrix of the previous layer.

$$\sum \text{tr}(\mathbf{Z}^T \times \mathbf{RM}) = \sum \text{tr}(\hat{\mathbf{Z}}).$$

Similarly, for the max-pooling layer, we set elements which correspond to the position of the largest element in each pooling region to "1" in the max-pooling operation matrix $\mathbf{PM}$. Then, the remaining elements in $\mathbf{PM}$ are set to "0", so that the max-pooling operations can be converted to the matrix multiplications

Algorithm 3: Improved zk-SNARK Based Proof Generation.**Input:**

Processed matrix $\mathbf{E}$;
 Feature matrix $\mathbf{Z}$;
 Common reference string crs ;
 Singular square matrix $\mathbf{U}$;
 Global model parameters θ ;

Output:

Proof π ;

- 1: Parse θ to obtain $\mathbf{W}$;
- 2: Parse $\mathbf{W}$ as $\{W_1, W_2, \dots, W_N\}$, $\mathbf{E}$ as $\{E_1, E_2, \dots, E_N\}$, and $\mathbf{Z}$ as $\{Z_1, Z_2, \dots, Z_N\}$;
- 3: Calculate subset commitment $Comm(\mathbf{E}) = \mathbf{U} \times \mathbf{E}$;
- 4: Set $L_i(x) = W_i \cdot L_{\mathbf{W}}(z) + \mathbf{U} \cdot L_{\mathbf{U}}(z)$,
 $R_i(x) = E_i \cdot R_{\mathbf{E}}(z)$,
 and $O_i(x) = Z_i \cdot O_{\mathbf{Z}}(z) + Comm(E_i) \cdot O_{\mathbf{C}}(z)$;
- 5: Execute $Prove(crs, \phi, \omega)$ as follows:
- 6: Parse the statement ϕ as $(Q_1, Q_2, \dots, Q_s)$;
- 7: Parse the witness ω as $(Q_{s+1}, Q_{s+2}, \dots, Q_N)$;
- 8: Parse crs as $(g^{\sigma_1}, h^{\sigma_2})$;
- 9: Randomly select $\psi, s \xleftarrow{\$} Z_p^*$;
- 10: Calculate
 $A = g^\alpha \cdot g^{\sum_{i=0}^N Q_i(z) L_i(x)} \cdot g^{\psi \delta}$
 $B = h^\beta \cdot h^{\sum_{i=0}^N Q_i(z) R_i(x)} \cdot h^{s \delta}$
 $C = \frac{\sum_{i=s+1}^N Q_i(z) (\beta L_i(x) + \alpha R_i(x) + O_i(x)) + h(x, z) t(x)}{\delta} + a s + b \psi - \psi s \delta$
 $D = g^{\frac{\sum_{i=0}^s Q_i(z) (\beta L_i(x) + \alpha R_i(x) + Z_i \cdot O_{\mathbf{Z}}(z))}{\gamma}}$
- 11: Get $\pi = (A, B, C, D, Comm(\mathbf{E}))$;
- 12: Return π .

of PM and $\hat{\mathbf{Z}}$.

$$\sum tr(\hat{\mathbf{Z}}^T \times PM) = \sum tr(\tilde{\mathbf{Z}}).$$

After that, the proof of the non-linear layer π_{act} and π_{pool} can be generated like π_{cov} via Algorithm 3.

Finally, the final trusted proof $\pi_{jr} = \{\pi_{cov}, \pi_{act}, \pi_{pool}\}$ is constructed by adopting CP-SNARKs [19] on the proofs of different layers. After the proof generation of the local model training in the r -th round, the P_j encrypts the model update $\tilde{g}_{jr}$ for the sake of privacy protection and generates an NIZK proof:

$$ZKP_{jr} \leftarrow NIZK(\tilde{g}_{jr}, \tilde{r}_j) \{Enc(\tilde{g}_{jr}, \tilde{r}_j) \wedge \|\tilde{g}_{jr}\|_2 < B\},$$

where $\tilde{r}_j$ is the masking key of the P_j and B is the norm bound set by the AS according to RoFL [33]. Due to the space limitations, we present the details of the ZKP_{jr} in our submitted *Supplemental Materials*. Then, P_j packages and signs the message $\{\pi_{jr}, Enc(\tilde{g}_{jr}, \tilde{r}_j), ZKP_{jr}\}$, and sends the message together with the signature to the AS.

D. Trustworthiness Verification

The AS first verifies the signature and parses the message to obtain the NIZK proof ZKP_{jr} . Then, the AS filters out the incorrect model updates if the proof ZKP_{jr} does not pass the

Algorithm 4: Trustworthiness Verification.**Input:**

Trusted proofs $\pi_{ji}, i \in \{1, \dots, T\}$;
 Common reference string crs and statement ϕ ;

Output:

True or False;

1: **for** $i = 1$ to T **do**

/*Data Authenticity Verification*/

2: Parse π_{ji} to obtain the commitment $Comm(\mathbf{E}_{ji})$;

3: **for** $k = 1$ to ξ **do**

4: Calculate

5: $bit_k = H_k(hash(Comm(\mathbf{E}_{ji}))) \bmod L$;

6: **while** $Bloom(bit_k) = 1$ **do**

7: Set $b_1 = 1$; continue;

8: **end while**

9: Set $b_1 = 0$; break;

10: **end for**

/*Training Integrity Verification*/

11: Parse π_{ji} to get $\{\pi_{cov}, \pi_{act}, \pi_{pool}\}$, where each proof is constructed in the form $(A, B, C, D, Comm(\mathbf{E}_{ji}))$

12: Verify $e(A, B) \stackrel{?}{=} e(g^\alpha, h^\beta) \cdot e(D, h^\gamma) \cdot e(C, h^\delta) \cdot e(g^{Comm(\mathbf{E}_{ji})}, h^\gamma)$ for each proof;

13: Set $b_2 = Verify(crs, \phi, \pi_{ji})$;

14: **end for**

15: $b = b_1 \oplus b_2$

16: If $b = 1$, then AS aggregates P_j 's gradient.

17: **When** $i = T$ **do**

/*Data Integrity Verification*/

18: Generate vectors $\vec{\eta} = (1, \eta^1, \eta^2, \dots, \eta^{n-1})^T$

19: **for** $i = 1$ to T **do**

20: $sum = \sum (Comm(\mathbf{E}_{ji}) \cdot \vec{\eta})$

21: $= \mathbf{U} \times \mathbf{E}_{j1} \cdot \vec{\eta} + \mathbf{U} \times \mathbf{E}_{j2} \cdot \vec{\eta} + \dots + \mathbf{U} \times \mathbf{E}_{jT} \cdot \vec{\eta}$

22: $= \mathbf{U} \times \mathbf{E}_j \cdot \vec{\eta}$

23: **end for**

24: **if** $Comm(\mathbf{E}_j) \cdot \vec{\eta} = sum$ **then**

25: Set $b_3 = 1$;

26: **else**

27: Set $b_3 = 0$;

28: **end if**

verification of the NIZK proof. Otherwise, the AS verifies the data authenticity and training integrity through the trusted proof π_{jr} to guarantee that the local model training has been executed trustworthily by using the committed dataset in the r -th round. When the required accuracy of the FL task is achieved, the data integrity verification is performed, as shown in Algorithm 4.

1) *Data Authenticity Verification*: Given the commitment $Comm(\mathbf{E}_{jr})$, the AS calculates the hash of the subset commitment with its ξ unbiased hash functions.

$$bit_k \leftarrow H_k(hash(Comm(\mathbf{E}_{jr}))) \bmod L,$$

where $k = 1, 2, \dots, \xi$. Each bit will be misjudged with the probability ϵ in the bloom filter. Suppose the function $Bloom(\cdot)$ determines the existence of a subset by querying whether the

corresponding bit of bit_k is "1", and AS queries the subset commitment successfully at each of ξ bits. In this case, there is a great probability of $1 - \epsilon^\xi$ to confirm that the commitment of this subset is in the bloom filter.

$$b_1 = bit_1 \oplus bit_2 \oplus \dots \oplus bit_\xi.$$

If $b_1 = 1$, it means that the subset used for local model training has been stored in the bloom filter before.

2) *Training Integrity Verification*: The AS verifies the trusted proof π_{jr} by executing the algorithm $Verify(crs, \phi, \pi_{jr})$. Specifically, the AS parses π_{jr} to get $\{\pi_{cov}, \pi_{act}, \pi_{pool}\}$, where each proof is constructed in the form $(A, B, C, D, Comm(\mathbf{E}_{ji}))$ and verifies the following equation for π_{cov} , π_{act} and π_{pool} .

$$e(A, B) = e(g^\alpha, h^\beta) \cdot e(D, h^\gamma) \cdot e(C, h^\delta) \cdot e(g^{\phi Comm(\mathbf{E}_{jr})}, h^\gamma)$$

If $b_2 = 1$, which means P_j passes the training integrity verification in the r -th round; Otherwise, $b_2 = 0$.

After the above verification, if $b_1 = b_2 = 1$, the encrypted model update $Enc(\vec{g}_{jr}, \vec{r}_j)$ will be accepted by the AS. And then, the AS aggregates the trustworthy model updates to obtain the encrypted global model $Enc(\theta_{r+1})$, which will be decoded [33] and broadcast for the next training round.

3) *Data Integrity Verification*: Assume the global model converges after t training rounds. In order to verify whether the P_j indeed uses all his claimed datasets D_{ji} , $i \in \{1, \dots, T\}$, the following three cases need to be considered.

Case I: when $t < T$, the federated training is completed before all committed subsets are consumed. It is required the AS asks all participants to submit their remaining subsets' commitments in the last round for integrity verification.

Case II: when $t = T$, the global model converges exactly when all committed datasets are exhausted. The AS performs data integrity verification based on the commitments of the subsets collected among T rounds.

Case III: when $t > T$, the global model remains short of requirements after all subsets have been trained. In this case, we proceed with model training to the next T -rounds using the same data subsets and continue to execute the integrity verification through Case I.

The AS can obtain $Comm(\mathbf{E}_{j1})$, $Comm(\mathbf{E}_{j2}), \dots, Comm(\mathbf{E}_{jT})$ from P_j , while the entire dataset $\mathbf{E}_j$ has been committed as $Comm(\mathbf{E}_j) = \mathbf{U} \times \mathbf{E}_j$. The AS can determine whether P_j uses the correct and entire subsets for the FL task by calculating $Comm(\mathbf{E}_j)\vec{\eta} = \sum (Comm(\mathbf{E}_{ji})\vec{\eta})$.

In case P_j uses only parts of the subsets to train the model, it can not pass the data integrity verification. Although the model convergence speed may be effected, our scheme can at least guarantee the correctness of the model updates aggregated in each training round.

V. SECURITY ANALYSIS

In this section, we revisit our design goals of TMT-FL presented in Section II-C and give a formal analysis accordingly.

Theorem 1 (Data Authenticity): The data authenticity of each training round is provided by the collision resistance of the hash function and the binding property of the commitment.

Proof: Since the malicious behaviors of participants will directly affect the data authenticity, we consider the following possible behaviors: 1) deviating from the local datasets preprocessing; 2) using poisoned datasets or parts of the datasets to train the local model. Assume a malicious participant P_A with the local dataset $\mathbf{D}_A$ deviates from the data preprocessing algorithm, and P_A converts local subset $\mathbf{D}_A$ into an incorrect matrix $\mathbf{E}'_A$ rather than $\mathbf{E}_A$. Then, at the next step (i.e., Trusted proof generation), the extracted feature matrix $\mathbf{Z}_A = f^{conv}(\mathbf{D}_A, \mathbf{W}_A)$ with $\mathbf{E}'_A$ cannot satisfy the multiplicative gates constraints, and then the proof can not be constructed, which is bad for P_A . So, there's no benefit for him to preprocess his dataset incorrectly. Hence, all participants including the malicious one will preprocess local datasets following Algorithm 2 for their own sake.

If the local dataset is poisoned after the preprocessing, the P_A performs the local training by using poisoned dataset or parts of the dataset $\mathbf{D}'_A$ and generates the local proof by using $\mathbf{E}'_A$ which is preprocessed from $\mathbf{D}'_A$ at round r . Here, we omit the training round r for notation simplicity. According to the system initialization step, the commitment of the dataset $Comm(\mathbf{E}_A)$ has already been stored in the bloom filter by the AS in advance. If P_A uses $\mathbf{D}'_A$ to train the local model, the commitment of $\mathbf{E}'_A$ can not be found in the bloom filter because of the binding property of the commitment, i.e., $Comm(\mathbf{E}'_A) \neq Comm(\mathbf{E}_A)$ and the collision-resistance of the hash functions $H_k(hash(Comm(.))) \bmod L$, $k \in \{1, 2, \dots, \xi\}$. If the local data has been poisoned before preprocessing and the participant has committed to the poisoned datasets, the local model update will be poisoned after local training. Under the circumstances, the P_A can pass the data authenticity verification, however, the incorrect model update will be filtered out by verifying the NIZK proof [33], which will not affect the accuracy of the global model convergence.

If the P_A trains the local model by using $\mathbf{D}'_A$ and generates the local proof $\pi_A = (A, B, C, D, Comm(\mathbf{E}'_A))$. P_A wants to trick the AS into passing the data authenticity verification, so P_A replaces $Comm(\mathbf{E}'_A)$ in π_A by using $Comm(\mathbf{E}_A)$. However, $Comm(\mathbf{E}_A)$, corresponding to the correct dataset $\mathbf{D}_A$, cannot satisfy the verification equation $e(A, B) \stackrel{?}{=} e(g^\alpha, h^\beta) \cdot e(D, h^\gamma) \cdot e(C, h^\delta) \cdot e(g^{\phi Comm(\mathbf{E}_A)}, h^\gamma)$, because $\mathbf{E}'_A$ is leveraged to generate the trusted proof rather than $\mathbf{E}_A$. This concludes the proof.

Theorem 2 (Training Integrity): The training integrity of TMT-FL relies on the trustworthiness of the proof, which is based on three properties of zk-SNARK, namely completeness, soundness, and zero-knowledge.

Lemma 2.1 (Completeness): If given true statement $\phi = (Q_1, Q_2, \dots, Q_s)$, a participant with the witness $\omega = (Q_{s+1}, Q_{s+2}, \dots, Q_N)$ can convince the AS that the participant has completed local training integrally.

$$Pr \left[Verify(crs, \phi, \pi) = 1 \mid \begin{matrix} (crs, td) \leftarrow Setup(R) \\ \pi \leftarrow Prove(crs, \phi, \omega) \end{matrix} \right] = 1.$$

Proof: Assume a participant P with a witness $\omega = (Q_{s+1}, Q_{s+2}, \dots, Q_N)$ generates a trusted proof $\pi = (A, B, C, D,$

$Comm(\mathbf{E})$) given the statement $\phi = (Q_1, Q_2, \dots, Q_s)$, according to Algorithm 3. Then the P submits the proof π and convinces AS by $Verify(\cdot)$ as follows.

$$\begin{aligned}
e(A, B) &= e(g^\alpha \cdot g^{Q_i(z)L_i(x)} \cdot g^{\psi\delta}, h^\beta \cdot h^{Q_i(z)R_i(x)} \cdot h^{s\delta}) \\
&= e(g, h)^{(\alpha+Q_i(z)L_i(x)+\psi\delta)(\beta+Q_i(z)R_i(x)+s\delta)} \\
&= e(g, h)^{\alpha\beta+\alpha Q_i(z)R_i(x)+\alpha s\delta+\beta Q_i(z)L_i(x)+s\delta Q_i(z)L_i(x)} \\
&\quad \cdot e(g, h)^{Q_i(z)^2 L_i(x)R_i(x)+\beta\psi\delta+\psi\delta Q_i(z)R_i(x)+\psi s\delta^2} \\
&= e(g, h)^{\alpha\beta} \cdot e(g, h)^{\frac{\sum_{i=0}^s Q_i(z)(\beta L_i(x)+\alpha R_i(x)+O_i(x))}{\gamma}} \cdot \gamma \\
&\quad \cdot e(g, h)^{\frac{\sum_{i=s+1}^N Q_i(z)(\beta L_i(x)+\alpha R_i(x))+h(x,z)t(x)}{\delta}} \cdot \delta \\
&\quad \cdot e(g, h)^{(as+b\psi-\psi s\delta)\delta} \\
&= e(g^\alpha, h^\beta) \cdot e(D, h^\gamma) \cdot e(C, h^\delta) \cdot e(g^{Comm(\mathbf{E})}, h^\gamma)
\end{aligned}$$

Since the participant needs to use the witness ω to calculate $p(x, z) = tr\{\mu \cdot \sum_{i=0}^N Q_i(z)L_i(x) \cdot \sum_{i=0}^N Q_i(z)R_i(x)\} - tr\{\mu \cdot \sum_{i=0}^N Q_i(z)O_i(x)\}$ when generating a correct proof π , and $p(x, z)$ must be divided by $t(x)$. Only the participant who has the witness $\omega = (Q_{s+1}, Q_{s+2}, \dots, Q_N)$ can construct the correct $p(x, z) = h(x, z)t(x)$ and make the above equation hold. This concludes the proof.

Lemma 2.2 (Soundness): A participant cannot prove a false statement, i.e., a participant cannot convince the AS that he has completed the local training with no witness. For any adversary $\mathcal{A}$ with the probabilistic polynomial time capability, it has

$$Pr \left[\begin{array}{c} Verify(crs, \phi, \pi) = 1 \\ \wedge (\phi, \omega) \notin R \end{array} \middle| \begin{array}{c} (crs, td) \leftarrow Setup(R) \\ (\phi, \pi, \omega) \leftarrow \mathcal{A}(R, crs, \tilde{a}x) \end{array} \right] \approx 0.$$

Proof: Assume an adversary P_A can extract a witness with a non-negligible success probability, and P_A can generate a proof as follows:

$$\begin{aligned}
A &= A_\alpha \alpha + A_\beta \beta + A_\gamma \gamma + A_\delta \delta + A(x, z) \\
&\quad + A_h(x, z) \frac{t(x)}{\delta} + \sum_{i=0}^s A_i \frac{\beta L_i(x) + \alpha R_i(x) + O_i(x)}{\gamma} \\
&\quad + \sum_{i=s+1}^N A_i \frac{\beta L_i(x) + \alpha R_i(x) + O_i(x)}{\delta},
\end{aligned}$$

given field elements $A_\alpha, A_\beta, A_\gamma, A_\delta, A_i$ and polynomials $A(x, z)$ and $A_h(x, z)$. Similarly, both B and C can be written as similar expressions for the proof. Since the verification equation can be viewed as equality of polynomials, the verification holds when regarding A, B , and C as formal polynomials generated by using $\alpha, \beta, \gamma, \delta, x, z$, according to the Schwartz-Zippel lemma [24].

The term $A_\alpha B_\alpha \alpha^2 = 0$ shows that $A_\alpha = 0$ or $B_\alpha = 0$. Without loss of generality, we can assume $B_\alpha = 0$ because $AB = BA$. The term $(A_\alpha B_\beta + A_\beta B_\alpha) \alpha \beta = \alpha \beta$ indicates that $A_\alpha B_\beta + A_\beta B_\alpha = A_\alpha B_\beta = 1$, which means $AB = (AB_\beta)(A_\alpha B)$, and we can assume $A_\alpha = B_\beta = 1$. Similarly, terms with β^2 show that $A_\beta B_\beta = A_\beta = 0$. Then A and B ,

constructed by the adversary, can be simplified to be the new form as follows:

$$\begin{aligned}
A &= \alpha + A_\gamma \gamma + A_\delta \delta + A(x, z) \\
&\quad + A_h(x, z) \frac{t(x)}{\delta} + \sum_{i=0}^s A_i \frac{\beta L_i(x) + \alpha R_i(x) + O_i(x)}{\gamma} \\
&\quad + \sum_{i=s+1}^N A_i \frac{\beta L_i(x) + \alpha R_i(x) + O_i(x)}{\delta} \\
B &= \beta + B_\gamma \gamma + B_\delta \delta + B(x, z) \\
&\quad + B_h(x, z) \frac{t(x)}{\delta} + \sum_{i=0}^s B_i \frac{\beta L_i(x) + \alpha R_i(x) + O_i(x)}{\gamma} \\
&\quad + \sum_{i=s+1}^N B_i \frac{\beta L_i(x) + \alpha R_i(x) + O_i(x)}{\delta}.
\end{aligned}$$

Then, considering the terms involving $\frac{1}{\delta^2}$, we have

$$\begin{aligned}
&(A_h(x, z)t(x) + \sum_{i=s+1}^N A_i(\beta L_i(x) + \alpha R_i(x) + O_i(x))) \\
&\quad \cdot (B_h(x, z)t(x) + \sum_{i=s+1}^N B_i(\beta L_i(x) + \alpha R_i(x) + O_i(x))) = 0.
\end{aligned}$$

Hence one of the terms in the left expression is 0. Because of the symmetry, we assume $A_h(x, z)t(x) + \sum_{i=s+1}^N A_i(\beta L_i(x) + \alpha R_i(x) + O_i(x)) = 0$. And then the terms in $\alpha \frac{B_h(x, z)t(x) + \sum_{i=s+1}^N B_i(\beta L_i(x) + \alpha R_i(x) + O_i(x))}{\delta} = 0$ show that $B_h(x, z)t(x) + \sum_{i=s+1}^N B_i(\beta L_i(x) + \alpha R_i(x) + O_i(x)) = 0$.

The terms with indeterminate $\frac{1}{\gamma^2}$ show that $(\sum_{i=0}^s A_i(\beta L_i(x) + \alpha R_i(x) + O_i(x))) \cdot (\sum_{i=0}^s B_i(\beta L_i(x) + \alpha R_i(x) + O_i(x))) = 0$. By symmetry, we can assume $\sum_{i=0}^s A_i(\beta L_i(x) + \alpha R_i(x) + O_i(x)) = 0$. Meanwhile, the terms in $\alpha \frac{\sum_{i=0}^s B_i(\beta L_i(x) + \alpha R_i(x) + O_i(x))}{\delta} = 0$ also show that $\sum_{i=0}^s B_i(\beta L_i(x) + \alpha R_i(x) + O_i(x)) = 0$.

The term $A_\gamma B_\gamma \gamma = 0$ implies $A_\gamma = 0$ and $B_\gamma \alpha \gamma = 0$ implies $B_\gamma = 0$. So, we have $A = \alpha + A_\delta \delta + A(x, z)$ and $B = \beta + B_\delta \delta + B(x, z)$.

In the verification equation, the remaining terms with α show us $\alpha B(x, z) = \sum_{i=0}^s Q_i(z) \alpha R_i(x) + \sum_{i=s+1}^N C_i(z) \alpha R_i(x)$, and the terms with β show that $\beta A(x, z) = \sum_{i=0}^s Q_i(z) \beta L_i(x) + \sum_{i=s+1}^N C_i(z) \beta L_i(x)$. Let us define $Q_i = C_i$ where $i = s+1, \dots, N$, we have $A(x, z) = \sum_{i=0}^s Q_i(z) L_i(x)$, $B(x, z) = \sum_{i=0}^s Q_i(z) R_i(x)$. Then, we can get the following equation by the terms with powers of x : $\sum_{i=0}^s Q_i(z) L_i(x) \cdot \sum_{i=0}^s Q_i(z) R_i(x) = \sum_{i=0}^s Q_i(z) O_i(x) + C_h(x, z)t(x)$. Since $Q_i(z)$ is irrelevant to $L_i(x), R_i(x), O_i(x)$ and $t(x)$, it means that $(Q_{s+1}(z), \dots, Q_N(z)) = (C_{s+1}(z), \dots, C_N(z))$ is a witness for the statement $(Q_1(z), \dots, Q_s(z))$. This concludes the proof.

Lemma 2.3 (Zero-Knowledge): The proof does not reveal anything but the truth of the statements, and the AS cannot distinguish whether the proof is generated by the participant

who performs the training process or the simulator.

$$\begin{aligned} & \Pr \left[\mathcal{A}(R, z, crs, td, \pi) = 1 \mid \begin{array}{l} (crs, td) \leftarrow \text{Setup}(R) \\ \pi \leftarrow \text{Prove}(crs, \phi, \omega) \end{array} \right] \\ &= \Pr \left[\mathcal{A}(R, z, crs, td, \pi) = 1 \mid \begin{array}{l} (crs, td) \leftarrow \text{Setup}(R) \\ \pi \leftarrow \text{Sim}(crs, td, \phi) \end{array} \right]. \end{aligned}$$

Proof: Assume Sim is an ideal simulator which can extract the trapdoor $td = (\alpha, \beta, \gamma, \delta, x, z)$, and a participant P_A with the trapdoor td can simulate proof π_A by executing algorithm $\pi_A \leftarrow \text{Sim}(crs, td, \phi)$. The P_A randomly chooses a, b, d, e to calculate a simulation proof $\pi_A = (g^a, h^b, g^c, g^d, g^e)$, where $c = \frac{ab - \alpha\beta - d\gamma - e\gamma}{\delta}$. The AS verifies the equation $e(g^a, h^b) \stackrel{?}{=} e(g^\alpha, h^\beta) \cdot e(g^d, h^\gamma) \cdot e(g^c, h^\delta) \cdot e(g^e, h^\gamma)$ using the proof π_A of the simulator. And the above equation always holds, which means that the verifier cannot distinguish the simulator from the provers by the verification equation. This concludes the proof.

Theorem 3 (Data Integrity): The data integrity of TMT-FL is based on the homomorphic and binding properties of the matrix commitment.

Proof: The AS can verify whether a participant has used the entire datasets in model training after T training rounds. Assume a participant P_A has repeatedly used only part of his claimed datasets $D_{A1}, D_{A2}, \dots, D_{Ar}$ ($r < T$) in local model training, and the corresponding processed datasets are $\mathbf{E}_{A1}, \mathbf{E}_{A2}, \dots, \mathbf{E}_{Ar}$. And the P_A asks for complete rewards after T training rounds. According to the data authenticity and training integrity verification, the r corresponding commitments $\text{Comm}(\mathbf{E}_{A1}), \text{Comm}(\mathbf{E}_{A2}), \dots, \text{Comm}(\mathbf{E}_{Ar})$ can be obtained from the trusted proofs $\pi_{A1}, \pi_{A2}, \dots, \pi_{Ar}$ of r training rounds. Then according to the data integrity verification algorithm, the AS constructs the vector $\vec{\eta} = (1, \eta^1, \eta^2, \dots, \eta^{n-m})^T$, where $\eta = \text{hash}(\text{Comm}(\mathbf{E}_A))$ has already stored at the AS side. And then, based on the homomorphic and binding property of the matrix commitment, the AS can calculate sum_A as follows:

$$\begin{aligned} & \text{sum}_A \\ &= \sum_{i=1}^r (\text{Comm}(\mathbf{E}_{Ai}) \vec{\eta}) \\ &= \text{Comm}(\mathbf{E}_{A1}) \vec{\eta} + \text{Comm}(\mathbf{E}_{A2}) \vec{\eta} + \dots + \text{Comm}(\mathbf{E}_{Ar}) \vec{\eta} \\ &= (\mathbf{U} \times \mathbf{E}_{A1}) \vec{\eta} + (\mathbf{U} \times \mathbf{E}_{A2}) \vec{\eta} + \dots + (\mathbf{U} \times \mathbf{E}_{Ar}) \vec{\eta} \\ &= \mathbf{U} \times (\mathbf{E}_{A1} + \mathbf{E}_{A2} + \dots + \mathbf{E}_{Ar}) \vec{\eta} \\ &= \mathbf{U} \times \left(\sum e_{i,1} \quad \sum e_{i,2} \quad \dots \quad \sum e_{i,n-m+1} \right) \cdot \vec{\eta} \\ &= \begin{bmatrix} u_{1,1} & \dots & u_{1,m^2} \\ u_{2,1} & \dots & u_{2,m^2} \\ \dots & \dots & \dots \\ u_{m^2,1} & \dots & u_{m^2,m^2} \end{bmatrix} \begin{bmatrix} e_{1,1} & \dots & e_{1,n-m+1} \\ e_{2,1} & \dots & e_{2,n-m+1} \\ \dots & \dots & \dots \\ e_{m^2,1} & \dots & e_{m^2,n-m+1} \end{bmatrix} \\ &= \begin{bmatrix} 1 \\ \eta \\ \dots \\ \eta^{n-m} \end{bmatrix} = \mathbf{U} \times (\mathbf{E}_A - \sum_{i=r+1}^T \mathbf{E}_{Ai} + \sum_{i=1}^r \mathbf{E}_{Ai}) \cdot \vec{\eta}. \end{aligned}$$

Since $\text{sum}_A \neq \text{Comm}(\mathbf{E}_A) \vec{\eta}$, where $\text{Comm}(\mathbf{E}_A) = \mathbf{U} \times \mathbf{E}_A$ has been stored by the AS in advance, so the P_A cannot pass the data integrity verification. Only the participant who has used all T subsets can pass the data integrity verification equation. This concludes the proof.

Theorem 4 (Resistance to Free-Riding Attacks): TMT-FL resists free-riding attacks through data authenticity and data integrity verification.

Proof: We consider three free-riding behaviors: 1) Submitting the same proof and local model update in different rounds; 2) Modifying the last round of model updates and submitting directly; 3) Intercepting other participants' data packets and submitting them to claim rewards.

Assume a malicious participant P_A submits the same proof π_A and local model update $\vec{g}$ in a different training round. Since TMT-FL requires participants to train with different subsets for each round, and data integrity verification is needed after the FL task. The AS can parse the proofs $\pi_{A1}, \pi_{A2}, \dots, \pi_{AT}$ and get the subsets commitments $\text{Comm}(\mathbf{E}_{A1}), \text{Comm}(\mathbf{E}_{A2}), \dots, \text{Comm}(\mathbf{E}_{AT})$. If the P_A submits the same proof π_{A1} in each round, then the subsets commitments $\text{Comm}(\mathbf{E}_{A1})$ will not pass data integrity verification according to Theorem 3.

In case the P_A modifies the last round of model update $\vec{g}$ with Gaussian noise and submits $\vec{g} + \vec{\varepsilon}$ to the AS directly. Since the P_A has not executed local model training, and his proof π_A will not pass training integrity verification. So, the incorrect model update $\vec{g} + \vec{\varepsilon}$ will not be aggregated to the global model. In case the P_A eavesdrops from the channel and gets the message $\{\pi_j, \text{Enc}(\vec{g}_j, \vec{r}_j), ZKP_j\}$ generated and transmitted by an honest participant P_j . The P_A attempts to submit them for rewards without any training efforts. Under this circumstance, the AS can authenticate the attribution of the message $\{\pi_j, \text{Enc}(\vec{g}_j, \vec{r}_j), ZKP_j\}$ via the signature and avoid handing out rewards to the P_A .

In conclusion, TMT-FL can resist free-riding attacks. This concludes the proof.

Theorem 5 (Privacy Preservation): TMT-FL preserves data privacy through the semantic security of the encryption algorithm $\text{Enc}()$ and the zero-knowledge properties of the NIZK proof ZKP and the zk-SNARK based proof π .

Proof: we present the formal proof via the simulation proof technique [26]. We first construct a simulator, which generates the entire view in the execution of TMT-FL, where the semi-honest server and some malicious participants are compromised by an adversary $\mathcal{A}$. Then, based on the simulator, we provide hybrid arguments to show that the real-world distribution is computationally indistinguishable from the simulated distribution. Since the simulator only uses the inputs and outputs of the $\mathcal{A}$, the indistinguishability means that the $\mathcal{A}$ cannot get anything useful beyond what they can derive from their inputs and prescribed outputs. Due to the space limitations, we present the details in our submitted *Supplemental Materials*.

VI. PERFORMANCE ANALYSIS

In this section, we first investigate the improved zk-SNARK based proof generation algorithm theoretically, and make

TABLE II
COMPARISON OF zk-SNARK BASED PROOF

Scheme	Proving Overhead
Keuffner [27] & VeriML [28]	$O(N \cdot M \cdot n^2 \cdot m^2)$
vCNN [15]	$O(N \cdot M \cdot (n^2 + m^2))$
pvCNN [29]	$O(N \cdot M \cdot n^2)$
TMT-FL	$O(N \cdot n^2)$

comparisons with the state-of-the-art zk-SNARK based schemes since TMT-FL is particularly proposed to address the trustworthy FL issue by using zk-SNARK based proof. Then, we conduct extensive experiments to evaluate the performance of TMT-FL in terms of computational cost, storage cost, communication overhead, and accuracy.

A. Theoretical Analysis of the zk-SNARK Based Proof

We evaluate our improved zk-SNARK based proof generation solution by comparing it to Keuffner [27], Zhao [28], vCNN [15], and pvCNN [29]. We use these four schemes as a baseline to illustrate how efficiency can be improved when generating a zk-SNARK based proof for the convolutional layer.

We set up the same environment for the comparison by applying a two-dimensional (2-D) convolution to a kernel of dimension $m \times m$ and a sample matrix of dimension $n \times n$. As shown in Table II, the column of proving overhead is for such 2-D convolution on N sample data along with M kernels. Keuffner [27] and Zhao [28] result in $O(N \cdot M \cdot n^2 \cdot m^2)$ proving overhead by directly applying Groth16. The vCNN [15] optimizes the proving overhead to $O(N \cdot M \cdot (n^2 + m^2))$ by converting matrix multiplications into polynomial operations and transforming the sum of polynomial products which represent the convolution into the product of polynomial accumulations. Then it eventually performs verifiable proofs in the Quadratic Polynomial Programs (QPP)-based zk-SNARK. The pvCNN [29] first considers batch proofs, and it separately flattens M kernels and N input matrices into a matrix of dimension $Mn^2 \times Mn^2$. For the sake of the efficiency, the number of inputs N is limited by the dimension of the input matrix $n \times n$, where $N \leq Mn^2$, and then it reduces the proving overhead to $O(N \cdot M \cdot n^2)$ by the batch matrix multiplication via QMP-based zk-SNARK.

Our solution reduces the proving time by separating the proof generation process into an offline preprocessing part (see Algorithm 2) and an online part (see Algorithm 3). The Chinese remainder theorem is used to preprocess the raw datasets to reduce the computational cost in convolution operations. Then we combine the batch convolution operations through an uncertain variable Q , such that the computational efficiency is optimized from proving the sum of matrix multiplication to the multiplication of matrix sums in the QMP-based zk-SNARK, bringing the online proving overhead to $O(N \cdot n^2)$.

B. Experimental Setup

In order to evaluate the performance of TMT-FL, the experiments are conducted over a 6-core intel processor, 7.8 GB RAM, and 40 GB Disk space, running Ubuntu 20.04. We use

TABLE III
COMPUTATIONAL COST OF CRT OPERATIONS

Number of polynomials	100	300	500	1000
CRT computation time	5.1973s	12.9672s	29.6721s	47.0699s
Inverse operation time	5.1963s	12.9641s	29.6655s	47.0520s
Montgomery for CRT	56.85ms	59.94ms	63.45ms	67.89ms

the libsnark library in C++ to implement the QMP-based and QAP-based zk-SNARK circuits. In addition, we leverage a cloud server of configuration RTX 2080Ti GPU, 62 GB RAM, and 100 GB Disk space for the local model training of federated learning. TMT-FL is evaluated with MNIST, Fashion-MNIST, CIFAR-10, and Tiny-Imagenet datasets. We set the number of kernels M as 1, 3, and 5, and the number of samples N in each subset as 1000, 2000, 3000, 4000, and 5000. The introduction of the datasets is as follows.

- *MNIST & Fashion-MNIST*: MNIST dataset is a handwritten digital image and corresponding labels, while the Fashion-MNIST dataset is an alternative to it and contains different kinds of fashion goods such as shirts, jackets, etc. Each consists of 60,000 training images and 10,000 test images, which are 28×28 pixel grayscale images. The images in the dataset are scaled to the range of 0 to 1 and are preprocessed and normalized.
- *CIFAR-10*: CIFAR-10 dataset contains 60,000 32×32 pixel color images in 10 classes, 50,00 of each class are used for training and 10,00 for testing. It has smaller images with more complex colors, and each pixel consists of 3 channels (red, green, and blue), thus making it more realistic and challenging than MNIST and Fashion-MNIST.
- *Tiny-Imagenet*: Tiny-Imagenet dataset is a subset of Imagenet, which comes from a course project at Stanford University. Tinyimagenet has 200 classes, each with 500 training samples and 50 test samples, each with the sample dimension of 64×64 of 3 channels.

C. Performance Evaluation

We first evaluate the performance of TMT-FL at the participant side and the AS side respectively at three phases: System initialization, Trusted proof generation, and Trustworthiness verification. Then, we investigate the training accuracy of TMT-FL on four real-world datasets.

1) *Cost at System Initialization*: a) *Computational cost*: At the participant side, the main operations in the system initialization step are the CRT operations, the commitments calculation, CRS generation, and the im2col transformation.

We first evaluate the computational cost of the CRT operations by setting a different number of remainder polynomials, i.e., 100, 300, 500, 1000. The computation time of CRT increases linearly with the number of the remainder polynomials. As shown in Table III, it takes 5.19 s, 12.97 s, 29.67 s, and 47.07 s when the number of remainder polynomials is 100, 300, 500, and 1000. Specifically, we find that the inverse operation of the modulus takes up the vast majority of computation time, so we adopt the Montgomery batch inversion to reduce the computational cost of the inversion operations.

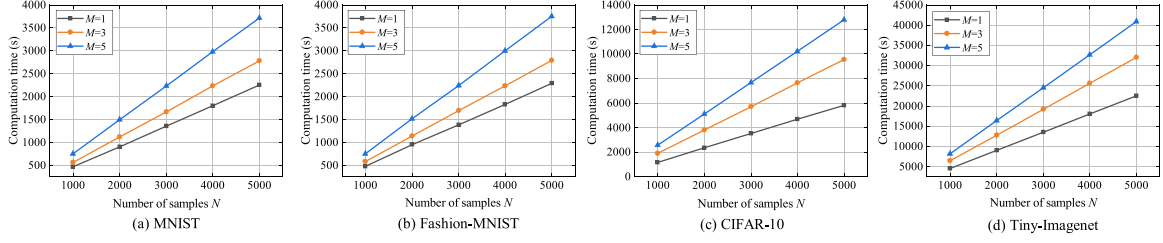


Fig. 5. Computational cost of the CRS generation on four datasets.

 TABLE IV
COMPUTATIONAL COST OF COMMITMENT CALCULATION

Datasets		MNIST	Fashion-MNIST	CIFAR-10	Tiny-Imagenet
Commitments					
N=1000	Pedersen comm.	404s	400s	600s	1720s
	Matrix comm.	2.78ms	2.20ms	2.90ms	5.98ms
N=3000	Pedersen comm.	1100s	1240s	1560s	4860s
	Matrix comm.	6.44ms	6.59ms	8.63ms	16.56ms
N=5000	Pedersen comm.	2220s	2360s	2660s	8460s
	Matrix comm.	12.17ms	12.74ms	13.90ms	26.85ms

We also evaluate the computational cost of the commitment calculation on four datasets and make a comparison between the matrix commitment [25] and Pedersen vector commitment. The kernel dimension $m \times m$ is set as 5×5 , $N=1000, 3000, 5000$, and the commitment dimension is set as 5×28 for MNIST and Fashion-MNIST, 5×32 for CIFAR-10, and 5×64 for Tiny-Imagenet. As shown in Table IV, the computation time unit is in seconds on Pedersen vector commitment and in milliseconds on matrix commitment. For different datasets, the computation time increases at different rates, which is positively related to the dimension of input matrices. Specifically, to compute 5000 commitments on Tiny-Imagenet datasets, it takes 8460 s to compute Pederson commitments, while matrix commitment calculation only takes 26.85 ms. Since the matrix dimension increases, Tiny-Imagenet requires more groups for Pedersen commitments and requires a larger square matrix U for matrix commitments.

As for the computational cost of the CRS generation, if we consider having a trusted third party generate the CRS, the computation time increases with the number of input samples linearly; meanwhile, it takes more time to generate the CRS when the number of kernels grows. In Fig. 5(a) and (b), the curve of the computation time is almost the same because the CRS generation process is only relevant to the dimension of the sample matrix which is 28×28 in both MNIST and Fashion-MNIST datasets. But the dimension of the sample matrix is different in CIFAR-10 and Tiny-Imagenet, which are 32×32 and 64×64 respectively. Hence, the computation time is different as shown in Fig. 5(a), (c), and (d). We also compare the computational cost of the CRS generation with Groth16 [13], vCNN [15], and pvCNN [29]. We set $M=5$, $m \times m = 5 \times 5$, $N=1000$, and perform experiments on different matrix dimension $n \times n$, i.e., $20 \times 20, 24 \times 24, 28 \times 28, 32 \times 32$. As shown in Fig. 6, when the matrix dimension is 32×32 , the computation time is 2.8x faster than vCNN and 2.6x faster than pvCNN.

Note that it takes about 2315 s to generate CRS on matrix dimension 64×64 in TMT-FL, while the auxiliary matrix that other schemes constructed for CRS generation has exceeded the

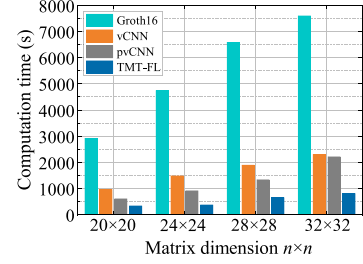


Fig. 6. Comparison of computational cost of CRS generation.

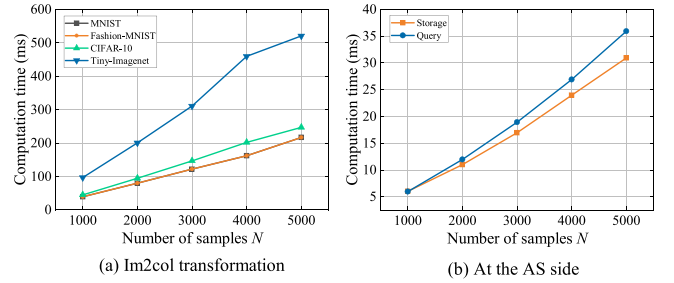


Fig. 7. Computational cost of im2col transformation and the operations at the AS side.

machine's caching capacity. When ρ participants generate the CRS collaboratively, the computational cost comprises the generation time of the CRS, the computation time of the verifiable secret sharing, and the time needed to aggregate parameters from the other $\rho-1$ participants.

Finally, we evaluate the computation time of the im2col transformation on four datasets, which is related to the matrix dimension of the dataset and the number of samples. In Fig. 7(a), The im2col transformation on dataset Tiny-Imagenet takes more time because the matrix dimension of Tiny-Imagenet is the largest, i.e., 64×64 . It needs about 500ms for 5000 samples in the Tiny-Imagenet.

At the AS side, AS needs to execute hash functions to keep the commitments of the dataset in the bloom filter. In addition, the AS queries the bloom filter to verify the authenticity and integrity of data subsets. The experimental results show that it takes little time to keep a subset in the bloom filter and query. As shown in Fig. 7(b), when the number of samples is 1000, 3000, and 5000, it takes only 6.00 ms, 16.97 ms, 30.92 ms for the storage of commitments and 5.96 ms, 18.92 ms, 35.42 ms for the verification of the query.

b) Storage cost: As for the storage cost, there are differences between the participant and the AS, both of which are related

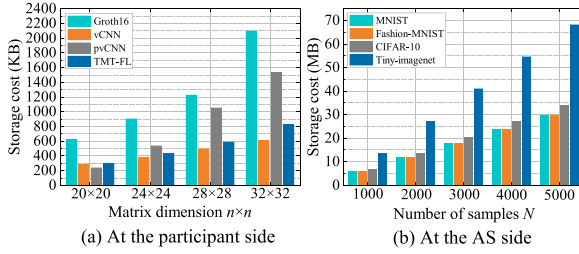


Fig. 8. Storage cost at the participant and AS side.

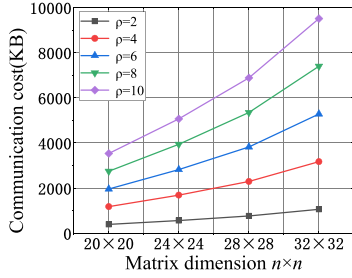


Fig. 9. Communication overhead for CRS generation.

to the matrix dimension. The participant needs to store CRS and some public parameters, such as modulus list and bilinear groups. Since the size of public parameters is much shorter than that of the CRS, and the modulus list takes up only 576B and 672B when applied to 28×28 and 32×32 matrix initialization, and the bilinear group takes only 100B. The storage cost at the participant side mainly depends on the size of CRS. We compare it with the CRS size of existing schemes. Fig. 8(a) shows that the storage cost varies in different settings, and the storage cost of TMT-FL is 1.3x bigger than vCNN and 0.5x smaller than pCNN when the matrix dimension is 32×32 .

The AS needs to maintain the bloom filter and the commitments of all participants. The storage cost at the AS side mainly depends on the size of the commitments generated by participants because the bloom filter takes up very little storage space. In our experiments, we assume that 10 participants are involved in an FL task, and the storage cost is 74.94 KB, 224 KB, and 374 KB for keeping 1000, 3000, and 5000 samples in the bloom filter respectively. In Fig. 8(b), it takes more space to keep the commitments of the datasets with a larger sample dimension. When the number of samples is 1000, 3000, 5000 in Tiny-imagenet, it takes 13.6 MB, 41.0 MB, and 68.2 MB for 10 participants.

c) Communication overhead: When the CRS is jointly generated by selected ρ participants, we evaluate the communication overhead with different matrix dimensions of samples in Fig. 9. For each participant, it is necessary to broadcast the verifiable secret shares of local parameters aggregated for the CRS generation, and it also needs to receive the broadcasts from other $\rho-1$ participants, which makes up the communication overhead for the CRS generation. When the matrix dimension $n \times n$ is set by 20×20 , 24×24 , 28×28 , 32×32 , it takes 3.44 MB, 4.94 MB, 6.71 MB, and 9.28 MB for 10 participants. And it is obvious that the more participants participate, the higher the communication overhead.

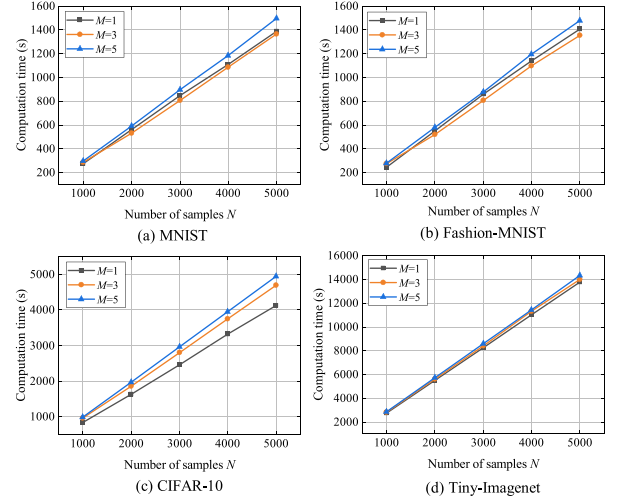


Fig. 10. Computational cost of proof generation for convolution.

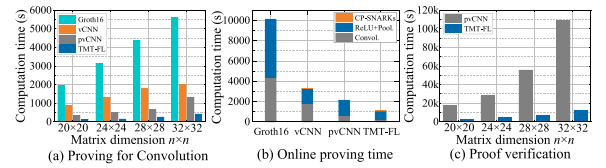


Fig. 11. Computational cost comparisons of proof generation and verification.

2) Cost at Trusted Proof Generation: The trusted proof generation is only executed by the participant. We first evaluate the computational cost of proof generation for the convolutional operations. As shown in Fig. 10(a) and (b), the computation time on MNIST and Fashion-MNIST is very close but remains subtle differences, because the matrix dimension of the data sample is the same in MNIST and Fashion-MNIST but the auxiliary polynomials constructed during the proving process is different.

Fig. 10(a), (c), and (d) show the computation time of proof generation on different datasets, i.e., MNIST, CIFAR-10, and Tiny-Imagenet, which has different matrix dimension. As the matrix dimension increases, there is a significant increase in the computation time of proof generation. In addition, by comparing with Fig. 5, we find that the setup time is more variable than the proving time in terms of the effect of the kernels, which means the proving time basically stays stable regardless of the number of kernels, especially in Fig. 10(d).

We also compare the computational cost of proof generation with some related works, i.e., Groth16, vCNN and pCNN. We set up 1,000 data samples for each participant. Since the Groth16 circuit cannot handle matrices directly, we convert the $n \times n$ -dimensional data sample to a polynomial of order n^2 , and convert 1000 matrix multiplication operations to the corresponding polynomial multiplication as the input to Groth16. In addition, according to the preprocessing structure of the vCNN and pCNN convolution operations, the input of the vCNN and pCNN convolution is also processed. Given the convolution kernel 5×5 and the dimension of the sample data 28×28 , we convert the input of vCNN to 1000 polynomial multiplications of order 808, and that of pCNN is converted to a large matrix multiplication of 3360×3360 . Fig. 11(a) shows

the comparison of the proving time for the convolutional layer of existing schemes. The empirical result shows that TMT-FL achieves that the proof generation for the convolutional layer is $6.8\times$ and $2.5\times$ faster than that of vCNN and pvCNN when the matrix dimension is 28×28 .

For the non-linear operations of the pooling and activation layer, we evaluate the proof generation performance using three methods, i.e., QAP-based method (adopted in vCNN and pvCNN), bitwise decomposition method (used in zkCNN [41]), and QMP-based method (our TMT-FL). Given the matrix dimension 28×28 , the empirical results show that the computation time for the QAP-based method is 1498.73s, the bitwise decomposition method takes 490s, and QMP-based method takes 404.6s. Then, we compare the total online proving time of the existing schemes, which contains the proving time for the convolutional layer, nonlinear layers and CP-SNARKs. Fig. 11(b) indicates that our TMT-FL has significant advantages and is $4.8\times$ and $3.1\times$ faster than vCNN and pvCNN on online proving time.

As for the NIZK proof, when using L_2 -norm proofs, the computational cost for proof generation comprises the creation of range proof and well-formedness proof. The well-formedness proof takes little time, so the range proof predominately determines the proving time, and it takes 157s for proof generation with a model update size of 262k. Meanwhile, it takes around 20s per participant for the verification of the NIZK proof [33].

3) *Cost at Trustworthiness Verification*: The AS performs the trustworthiness verification operation, which involves three verification operations: i.e., data authenticity, data integrity, and training Integrity. Data authenticity is achieved through the query of the bloom filter, while the verification cost is equal to the query time of the bloom filter. Data integrity is achieved by verifying the commitment equation, of which the verification time is negligible. We compare the performance of the training integrity verification with pvCNN. As shown in Fig. 11(c), the computational cost of TMT-FL is an order of magnitude smaller than that of pvCNN. We find that the verification time of TMT-FL and pvCNN is measured in seconds, because the QMP-based proof verification is calculated on an arithmetic circuit over matrices.

4) *Accuracy of the Proposed TMT-FL*: In this section, we mainly analyze the impact of different behaviors of malicious participants on accuracy, including training with poisoned data or partial datasets and submitting incorrect model updates to the AS. In our experiments, we assume there are 10 participants and one AS. The training task on four real datasets is to train a CNN model, and the training data is divided evenly into 10 parts.

We set the expected training rounds T as 10. For MNIST, each participant is assigned 6000 training images with a learning rate of 0.01 and 10 epochs per round. For CIFAR-10, each participant is assigned 5000 training images, and the learning rate is set as 0.001. For Tiny-Imagenet, we divide the dataset independently and identically and apply the Resnet18 network model for federated training.

In Fig. 12(a), (b), (c), there are two participants to be malicious in our setting which use partial datasets (25%, 50%, 75% of the entire dataset) to train local model, and the results show the effect of using partial datasets on the convergence speed. It is the fastest for the global model to converge where each participant uses the

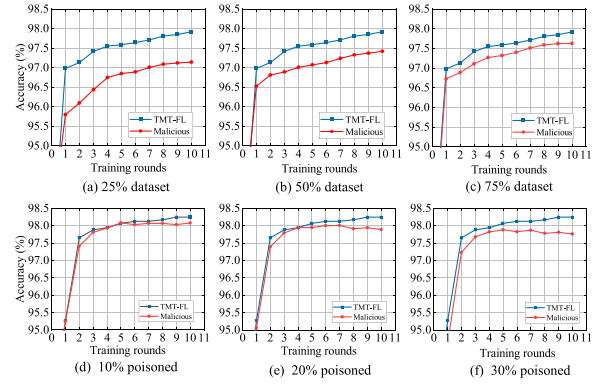


Fig. 12. Effect of partial/poisoned datasets on accuracy.

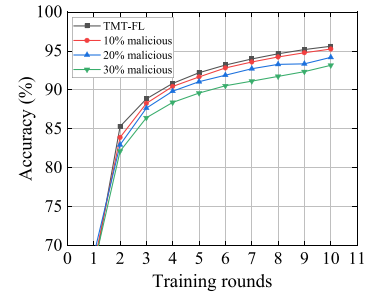


Fig. 13. Effect of the incorrect model on accuracy.

entire dataset. TMT-FL guarantees that participants train with the entire dataset, otherwise they will not pass the data integrity verification algorithm and waste their time and resources. As demonstrated in Fig. 12(d), (e), (f), it is obvious that using poisoned datasets affects the convergence of the global model. In our setting, there are two malicious participants using datasets with different poisoned rates, i.e., 10%, 20%, 30%. The verification of the NIZK proof guarantees that the model updates trained by using the poisoned data will not be aggregated in the global model, which will not affect the training accuracy. Obviously, the more poisoned data used, the greater the impact on accuracy.

In Fig. 13, we set different proportions of participants modifying their local gradients, i.e., 10%, 20%, 30%, and submitting incorrect model updates to the AS. TMT-FL can guarantee a quick convergence of the global model because the incorrect model fails to pass the verification equation during the training integrity verification so that they will not affect the accuracy. The poisoned local model affects the global model convergence depending on the proportion of the incorrect model updates.

VII. RELATED WORK

In this section, we review previous studies relevant to our work in two categories, i.e., secure federated learning and verifiable federated learning.

Secure Federated Learning: Since FL requires that the raw data is stored locally and can not be transferred, the limited visibility of the local training process makes FL vulnerable to security attacks, such as data/model poisoning attacks and free-riding attacks. As for data poisoning attacks, Shen et al. [5] proposed to cluster local updates and remove outliers to ensure the authenticity of training data. Fung et al. [30] addressed

the Sybil-based datasets poisoning by measuring the angular distance of gradients via cosine similarity. Andreina et al. [31] incorporated feedback loops into FL to validate the correctness of model parameters through voting, which can resist datasets with backdoors. Zhao et al. [7] employed a functioning mechanism to perturb the objective function of the neural network during the training process, achieving ϵ -differential privacy. They utilized the exponential mechanism to remove malicious datasets of unreliable participants under the protection of model privacy.

As for the model poisoning attacks, EIFFeL [32] used secret-shared non-interactive proofs (SNIP) and verifiable secret sharing (VSS) to ensure input completeness. It verifies the gradient completeness without violating the participant's privacy and ultimately achieves secure aggregation. RoFL [33] achieved secure federated learning with robustness by combining homomorphism encryption (HE) and mask-based secure aggregation methods, and realized it via zero-knowledge proofs (ZKP). For free-riding attacks, Lin et al. [34] proposed a high-dimensional anomaly detection method, which combined the standard deviation measure of locally updated gradient matrices with the deep autoencoding Gaussian mixture model. The above schemes achieve different security goals, however, they do not consider whether real datasets are used in the model training or whether the local training process is correctly executed, which makes it incapable of defending against advanced stealth attacks and free-riding behaviors from malicious participants.

Verifiable Federated Learning: Many verifiable federated learning schemes have been proposed to verify the correctness of either the aggregation results of the server or the training execution of the participants in a malicious threat model. A malicious server may falsify the aggregation results deliberately or unintentionally, which will affect the global model convergence. To address this issue, some studies [35], [36], [37], [38] achieved gradient aggregation verification by introducing various technologies, such as additive homomorphism, random matrix encoding, zero-knowledge proofs, etc. Meanwhile, malicious participants may deviate from the local training process which results in incorrect model updates. To prove the trustworthiness of local execution process, existing solutions mainly include TEE-based and zk-SNARK proof-based schemes. Zhang et al. [11] proposed TrustFL to ensure proper training execution via TEE and commitments. Chen et al. [39] designed a training integrity protocol based on TEE to ensure the completeness of the deep learning process, guaranteeing the trustworthiness of local model training and model aggregation. Tang et al. [40] made confidential gradients verifiable by using Paillier and ZKPs under the Multilayer Perceptron scenario.

As for zk-SNARK proof-based solution, few efforts have explored this problem in FL. Some other works related to zk-SNARK give us some inspiration. Zhao et al. [28] optimized SNARK arithmetic circuits and provided integrity for local training to ensure that participants can correctly execute tasks on untrusted servers. Wan et al. [10] bound the arithmetic circuits of zk-SNARK with hash values to meet data consistency between proof and training. Lee et al. [15] proposed a novel and efficient verifiable CNN and optimized the convolutional operations by converting matrix multiplication into polynomial calculations.

Liu et al. [41] reduced the two-dimensional convolution to one-dimensional through the Fast Fourier Transform, and constructed a new sum-check protocol to minimize the proof generation time. Feng et al. [42] proposed a Rank-1 constraint system quantization method for privacy protection on CNNs and put it on encoding to achieve an optimizing compiler for verifiable local training. Duan et al. [43] constructed a zero-knowledge proof scheme by converting computations into custom gate within the circuit, and achieved a lightweight neural network backpropagation verification through the multi-linear sum-check protocol. Weng et al. [29] constructed a new arithmetic circuit based on the quadratic matrix program, on which it proves batch convolutional computation in the form of matrix multiplication for model training.

In this paper, we start from improving the classical zk-SNARK and apply it together with some technologies to guarantee the training integrity, data authenticity, data integrity and privacy protection in an FL task with malicious participants. Table V compares TMT-FL with prior works.

VIII. DISCUSSIONS

Proof Generation Based on zk-SNARK: In TMT-FL, we generate local proofs via QMP-based zk-SNARK to guarantee the training integrity at each layer of CNN. Due to the complexity of neural networks which consists of multiple linear and non-linear operations, the efficiency of generating proofs needs to be improved. Especially for non-linear operations such as pooling and activation layers, existing schemes usually use average pooling rather than max pooling, or they transform the non-linear activation function to the linear one by using some linear fitting methods. These solutions will make an impact on the model's accuracy. Recently, vCNN [15] and zkCNN [41] have adopted bitwise operations to handle the max-pooling and ReLU operations. However, it is not applicable to directly adopt this approach of dealing with non-linear operations to our TMT-FL. Because the batch matrix multiplication processed by TMT-FL will result in an exponential increase in the number of bitwise operations. We convert non-linear operations in CNNs into matrix operations by constructing 0/1 auxiliary matrices without affecting the accuracy. And the efficiency of generating proof based on zk-SNARK has been improved compared to existing schemes. Nevertheless, it deserves further study.

Data Integrity Verification of TMT-FL: To accomplish data integrity verification, we propose a strategy that handles the division of local subsets corresponding to the training rounds. Since in each round of local training, the training subset is composed of randomly selected training samples, this leads to difficulties in determining the correspondence between the training subset and the entire dataset. Therefore, we assumed the number of rounds for global model convergence and partitioned the dataset equally based on the number of rounds, which is contrary to the convention of traditional machine learning where the training subset is constructed by selecting data samples randomly. Therefore, it is our future work to design optimal strategies to accomplish data integrity verification.

Realization of the Trustworthiness: We adopt zk-SNARK based proof to ensure trustworthy model training, where proofs

TABLE V
COMPARISON OF FUNCTIONALITY WITH THE MOST RELEVANT WORKS

Scheme	Participant setting	Server setting	Correctness for model updates	Privacy for local models	Integrity for local training	Data authenticity	Data Integrity	Underlying technology
EIFFeL [32]	malicious	malicious	✓	✓	×	×	×	SNIP+VSS
RoFL [33]	malicious	semi-honest	✓	✓	×	×	×	HE+ZKP
TrustFL [11]	dishonest	semi-honest	✓	×	✓	×	×	TEE
Chen et al. [39]	malicious	malicious	✓	✓	✓	×	×	TEE
PILE [40]	malicious	semi-honest	✓	✓	✓	×	×	HE+ZKP
TMT-FL (Our)	malicious	semi-honest	✓	✓	✓	✓	✓	zk-SNARKs

are generated individually by each participant and are publicly verifiable. Although our system model is based on a Client/Server architecture, the proof generation algorithm can also be applied to a decentralized architecture. In a decentralized architecture combined with blockchain, it is possible to use our algorithm directly. In a device-to-device architecture, we can modify our proof generation algorithm by introducing embedded proof techniques [27]. This is our work in progress. In addition, we also explore the advantages of zk-SNARK based proof and TEE-based solution and consider the combination of them to better achieve trustworthy FL.

Privacy Protection of Participants: In FL, it is possible to infer sensitive data distributions from local model updates, resulting in privacy leakage [44]. For the privacy protection requirements of the model updates, they can be well addressed by HE, differential privacy, random masking and secure multi-party techniques [45], [46], [47]. However, encrypted/masked model updates provide concealment for poisonous model updates, which may easily circumvent most existing defense strategies. The existing solutions [32], [33], [48] are mainly based on two-server or single-server architecture, where single-server solutions are more challenging. In order to meet the requirements of the single server architecture, we adopt the NIZK proof [33] to help filter out incorrect model updates from the encrypted model updates. Nevertheless, how to resist encrypted/masked model poisoning attacks deserves further studies.

IX. CONCLUSION

In this paper, we studied the trustworthy problem of model training of FL in the presence of malicious participants. We proposed TMT-FL, a trustworthy model training scheme that guarantees both the authenticity and integrity of the data and model training. We presented an offline-then-online strategy based on the Chinese Remainder Theorem and batch proofs to optimize the proof generation time of convolution in QMP-based zk-SNARK. Moreover, we combined matrix commitments with bloom filters to achieve the authenticity and integrity verification of the participant's local dataset. Finally, we experimentally evaluated the performance of TMT-FL under four datasets, i.e., MNIST, Fashion-MNIST, CIFAR-10, and Tiny-Imagenet, demonstrating that TMT-FL has advantages in proving time and scalability compared with existing schemes.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [2] W. Zhang et al., "Dynamic-fusion-based federated learning for COVID-19 detection," *IEEE Internet Things J.*, vol. 8, no. 21, pp. 15884–15891, Nov. 2021.
- [3] S. R. Pokhrel and J. Choi, "A decentralized federated learning approach for connected autonomous vehicles," in *Proc. 2020 IEEE Wireless Commun. Netw. Conf. Workshops*, 2020, pp. 1–6.
- [4] D. Kawa, S. Punyani, P. Nayak, A. Karkera, and V. Jyotinagar, "Credit risk assessment from combined bank records using federated learning," *Int. Res. J. Eng. Technol.*, vol. 6, no. 4, pp. 1355–1358, 2019.
- [5] S. Shen, S. Tople, and P. Saxena, "AUROR: Defending against poisoning attacks in collaborative deep learning systems," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, 2016, pp. 508–519.
- [6] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 119–129.
- [7] L. Zhao, Q. Wang, Q. Zou, Y. Zhang, and Y. Chen, "Privacy-preserving collaborative deep learning with unreliable participants," *IEEE Trans. Inf. Forensics Security*, vol. 15, no. 9, pp. 1486–1500, 2019.
- [8] Z. Zhang, Y. Zhang, D. Guo, L. Yao, and Z. Li, "SecFedNIDS: Robust defense for poisoning attack against federated learning-based network intrusion detection system," *Future Gener. Comput. Syst.*, vol. 134, pp. 154–169, 2022.
- [9] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Proc. Annu. Int. Cryptol. Conf.*, 1991, pp. 129–140.
- [10] Z. Wan, Y. Zhou, and K. Ren, "zk-AuthFeed: Protecting data feed to smart contracts with authenticated zero-knowledge proof," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 1335–1347, Mar./Apr. 2023.
- [11] X. Zhang, F. Li, Z. Zhang, Q. Li, C. Wang, and J. Wu, "Enabling execution assurance of federated learning at untrusted participants," in *Proc. 2020 IEEE Conf. Comput. Commun.*, 2020, pp. 1877–1886.
- [12] L. Wang, X. Zhao, Z. Lu, L. Wang, and S. Zhang, "Enhancing privacy preservation and trustworthiness for decentralized federated learning," *Inf. Sci.*, vol. 628, pp. 449–468, 2023.
- [13] J. Groth, "On the size of pairing-based non-interactive arguments," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2016, pp. 305–326.
- [14] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, and D. X. Song, "Libra: Succinct zero-knowledge proofs with optimal prover computation," in *Proc. Annu. Int. Cryptol. Conf.*, 2019, pp. 733–764.
- [15] S. Lee, H. Ko, J. Kim, and H. Oh, "vCNN: Verifiable convolutional neural network based on zk-SNARKs," *IEEE Trans. Inf. Forensics Security*, vol. 21, no. 4, pp. 4254–4270, Aug. 2024.
- [16] J. Zhang et al., "Doubly efficient interactive proofs for general arithmetic circuits with linear prover time," in *Proc. 2021 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 159–177.
- [17] D. Pei, A. Salomaa, and C. Ding, *Chinese Remainder Theorem: Applications in Computing, Coding, Cryptography*. Singapore: World Scientific, 1996, pp. 1–213.
- [18] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [19] M. Campanelli, D. Fiore, and A. Querol, "LegoSNARK: Modular design and composition of succinct zero-knowledge proofs," in *Proc. 2019 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 2075–2092.
- [20] P. Feldman, "A practical scheme for non-interactive verifiable secret sharing," in *Proc. 28th Annu. Symp. Found. Comput. Sci.*, 1987, pp. 427–438.
- [21] K. Chellapilla, S. Puri, and P. Simard, "High performance convolutional neural networks for document processing," in *Proc. 10th Int. Workshop Front. Handwriting Recognit.*, 2006, pp. 1–6.
- [22] A. Francesco and G. Sebastian, "Method for verifying information," 2021. [Online]. Available: <https://patentimages.storage.googleapis.com/53/3c/62/0ed0b3f9bb163f/US20210091953A1.pdf>
- [23] Y. Fan, B. Xu, L. Zhang, J. Song, A. Zomaya, and K. Li, "Validating the integrity of convolutional neural network predictions based on zero-knowledge proof," *Inf. Sci.*, vol. 625, pp. 125–140, 2023.

- [24] S. Bayer and J. Groth, "Efficient zero-knowledge argument for correctness of a shuffle," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2012, pp. 263–280.
- [25] S. Gao, J. Luo, J. Zhu, X. Dong, and W. Shi, "VCD-FL: Verifiable, collusion-resistant, and dynamic federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, no. 4, pp. 3760–3773, Apr. 2023.
- [26] Y. Lindell, "How to simulate it-A tutorial on the simulation proof technique," in *Tutorials on the Foundations of Cryptography*, Cham, Switzerland: Springer, 2017, pp. 277–346.
- [27] J. Keuffer, R. Molva, and H. Chabanne, "Efficient proof composition for verifiable computation," in *Proc. 23rd Eur. Symp. Res. Comput. Secur.*, 2018, pp. 152–171.
- [28] L. Zhao, Q. Wang, C. Wang, Q. Li, C. Shen, and B. Feng, "VeriML: Enabling integrity assurances and fair payments for machine learning as a service," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 10, pp. 2524–2540, Oct. 2021.
- [29] J. Weng, J. Weng, G. Tang, A. Yang, M. Li, and J. Liu, "pvCNN: Privacy-preserving and verifiable convolutional neural network testing," *IEEE Trans. Inf. Forensics Security*, vol. 18, no. 3, pp. 2218–2233, Mar. 2023.
- [30] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "Mitigating sybils in federated learning poisoning," 2018, *arXiv: 1808.04866*.
- [31] S. Andreina, G. A. Marson, H. Möllering, and G. Karame, "BaFFLe: Backdoor detection via feedback-based federated learning," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 852–863.
- [32] A. Chowdhury, C. Guo, S. Jha, and L. Maaten, "EIFFeL: Ensuring integrity for federated learning," in *Proc. 2022 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2535–2549.
- [33] H. Lycklama, L. Burkhalter, A. Viand, N. Küchler, and A. Hithnawi, "RoFL: Robustness of secure federated learning," in *Proc. 2023 IEEE Symp. Secur. Privacy*, 2023, pp. 453–476.
- [34] J. Lin, M. Du, and J. Liu, "Free-riders in federated learning: Attacks and defenses," 2019, *arXiv: 1911.12560*.
- [35] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 15, no. 7, pp. 911–926, Jul. 2020.
- [36] Z. Wang, N. Dong, J. Sun, W. Knottenbelt, and Y. Guo, "zkFL: Zero-knowledge proof-based gradient aggregation for federated learning," *IEEE Trans. Big Data*, pp. pp. 1–14, May 2024, doi: [10.1109/TB-DATA.2024.3403370](https://doi.org/10.1109/TB-DATA.2024.3403370).
- [37] H. Zhou, G. Yang, Y. Huang, H. Dai, and Y. Xiang, "Privacy-preserving and verifiable federated learning framework for edge computing," *IEEE Trans. Inf. Forensics Security*, vol. 18, no. 12, pp. 565–580, Dec. 2023.
- [38] Z. Zhang et al., "G-VCFL: Grouped verifiable chained privacy-preserving federated learning," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4219–4231, Dec. 2022.
- [39] Y. Chen, F. Luo, T. Li, T. Xiang, Z. Liu, and J. Li, "A training-integrity privacy-preserving federated learning scheme with trusted execution environment," *Inf. Sci.*, vol. 522, pp. 69–79, 2020.
- [40] X. Tang, M. Shen, Q. Li, L. Zhu, T. Xue, and Q. Qu, "PILE: Robust privacy-preserving federated learning via verifiable perturbations," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 5005–5023, Nov./Dec. 2023.
- [41] T. Liu, X. Xie, and Y. Zhang, "zkCNN: Zero-knowledge proofs for convolutional neural network predictions and accuracy," in *Proc. 2021 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 2968–2985.
- [42] B. Feng, L. Qin, Z. Zhang, Y. Ding, and S. Chu, "ZEN: An optimizing compiler for verifiable, zero-knowledge neural network inferences," *Cryptol. ePrint Arch.*, Paper 2021/087, 2021. [Online]. Available: <https://eprint.iacr.org/2021/087>
- [43] H. Duan, Z. Peng, L. Xiang, Y. Hu, and B. Li, "A verifiable and privacy-preserving federated learning training framework," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 5, pp. 5046–5058, Sep./Oct. 2024.
- [44] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 14774–14784.
- [45] C. Dong et al., "Privacy-preserving and Byzantine-robust federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 889–904, Mar./Apr. 2024.
- [46] Y. Zhao and H. Sun, "Information theoretic secure aggregation with user dropouts," *IEEE Trans. Inf. Theory*, vol. 68, no. 11, pp. 7471–7484, Nov. 2022.
- [47] J. Zhao, H. Zhu, F. Wang, R. Lu, Z. Liu, and H. Li, "PVD-FL: A privacy-preserving and verifiable decentralized federated learning framework," *IEEE Trans. Inf. Forensics Security*, vol. 17, no. 5, pp. 2059–2073, May 2022.
- [48] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, "ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 17, no. 4, pp. 1639–1654, Apr. 2022.



Zhongkai Lu received the master's degree from the Qingdao University of Science and Technology, Qingdao, China, in 2024. He is currently working toward the PhD degree with the School of Cyber Engineering, Xidian University, Xi'an, Shaanxi, China. His research interests include trustworthy federated learning, applied cryptography, and zero-knowledge proof.



Lingling Wang (Member, IEEE) received the PhD degree from Harbin Engineering University, Harbin, China, in 2008. She is an associate professor with the School of Information Science and Technology, Qingdao University of Science and Technology, Qingdao, China. She was sponsored by China Scholarship Council (CSC) as a visiting scholar with Wilfrid Laurier University, Waterloo, ON, Canada, in 2019. Her research interests include AI security and applied cryptography.



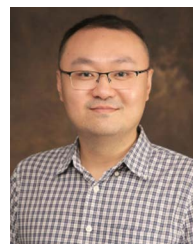
Zhengyin Zhang received the BE degree from Shandong Xiehe University, Jinan, China, in 2022. He is currently working toward the master degree with the School of Information Science and Technology, Qingdao University of Science and Technology, Qingdao, China. His research interests include federated learning and privacy protection.



Mei Huang received the BE degree from the Qingdao University of Science and Technology, Qingdao, China, in 2022. She is currently working toward the master degree with the School of Information Science and Technology, Qingdao University of Science and Technology, Qingdao, China. Her research interests include federated learning and privacy protection.



Jingjing Wang (Member, IEEE) received the PhD degree in computer application technology from the Ocean University of China, Qingdao, China, in 2012. She is a professor with the School of Information Science and Technology, Qingdao University of Science and Technology (QUST). She was a visiting professor with the University of British Columbia from 2014 to 2015. Her research interests include wireless communications, bionic communications, and secure data transmission.



Meng Li (Senior Member, IEEE) received the PhD degree in computer science and technology from the Beijing Institute of Technology (BIT), China, in 2019. He is an associate professor with the School of Computer Science and Information Engineering, Hefei University of Technology, China. He was sponsored by ERCIM 'Alain Bensoussan' Fellowship Programme to conduct post-doc research. He was sponsored by China Scholarship Council (CSC) for joint PhD study with the University of Waterloo and Wilfrid Laurier University. His research interests include applied cryptography, edge computing, blockchain, and vehicular networks.